

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Интеллектуальная система для выявления заболеваний

сельскохозяйственных растений

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

(подпись, дата)

Н. М. Крюков

(инициалы, фамилия)

Группа ПО-116

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

(инициалы, фамилия)

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

(подпись, дата)

А. В. Малышев

(инициалы, фамилия)

Курск 2025 г.

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

« ____ » _____ 20 ____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА

Студента Крюкова Н. М., шифр 21-06-0071, группа ПО-116

1. Тема «Интеллектуальная система для выявления заболеваний сельскохозяйственных растений» утверждена приказом ректора ЮЗГУ от «04» апреля 2025 г. № 1696-с.

2. Срок предоставления работы к защите «09» июня 2025 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

- 1) провести анализ предметной области;
- 2) разработать концептуальную модель интеллектуальной системы для выявления заболеваний томатов;
- 3) спроектировать модель нейронной сети и веб-приложения;
- 4) реализовать и протестировать программную систему.

3.2. Входные данные и требуемые результаты для программы:

- 1) Входными данными для программной системы являются изображения кустов или листьев томатов.
- 2) Выходными данными для программной системы являются: информация о наличии заболевания, его описание, рекомендации по лечению и профилактике.

4. Содержание работы (по разделам):

4.1. Введение.

4.1. Анализ предметной области.

4.2. Техническое задание: основание для разработки, цель и назначение разработки, актуальность темы разработки, этапы разработки, требования к программной системе, требования к оформлению документации.

4.3. Технический проект: общие сведения о программной системе, обоснование выбора технологий проектирования, проектирование архитектуры программной системы, структура модулей программы, структура базы данных.

4.4. Рабочий проект: спецификация компонентов и классов программной системы, обучение нейронной сети, тестирование программной системы.

4.5. Заключение.

4.6. Список использованных источников.

5. Перечень графического материала:

Лист 1. Сведения о ВКРБ.

Лист 2. Цель и задачи разработки.

Лист 3. Диаграмма прецедентов.

Лист 4. Диаграмма компонентов.

Лист 5. Архитектура YOLO 11.

Лист 6. Структура модулей.

Лист 7. ER-диаграмма.

Лист 8. Динамика метрик обучения.

Лист 9. Кривые F1-меры в зависимости от порога уверенности.

Лист 10. Кривые точности и полноты.

Лист 11. Нормализованная матрица ошибок.

Лист 12. Результат анализа фотографии.

Лист 13. Заключение.

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

Н. М. Крюков

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 102 страницам. Работа содержит 44 иллюстрации, 9 таблиц, 54 библиографических источников и 13 листов графического материала. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: интеллектуальная система, компьютерное зрение, сверточные нейронные сети, YOLOv11, Flask, PostgreSQL, веб-приложение, диагностика заболеваний, сельскохозяйственные растения, томаты, база данных, модуль, компонент, пользовательский интерфейс, машинное обучение, обработка изображений, Python, Google Colab, PyTorch, информационные технологии.

Объектом разработки является интеллектуальная система для диагностики заболеваний сельскохозяйственных растений, в частности томатов, с использованием технологий компьютерного зрения и машинного обучения.

Целью выпускной квалификационной работы является повышение эффективности диагностики заболеваний сельскохозяйственных растений путем создания интеллектуальной системы, которая позволяет загружать изображения растений, выявлять заболевания и предоставлять рекомендации по их лечению и профилактике.

В процессе создания системы были выделены основные сущности путем проектирования базы данных, использованы модули и классы, обеспечивающие обработку изображений и взаимодействие с базой данных, разработан пользовательский интерфейс на основе HTML/CSS и фреймворка Flask, а также реализована модель машинного обучения YOLOv11 для анализа изображений растений и выявления заболеваний.

При разработке системы использовались фреймворк Flask для серверной части, PostgreSQL для хранения данных о болезнях и рекомендациях, а также Python и PyTorch для реализации и обучения модели машинного обучения.

Разработанная система была успешно протестирована.

ABSTRACT

The volume of work is 102 pages. The work contains 44 illustrations, 9 tables, 54 bibliographic sources and 13 sheets of graphic material. The number of applications is 2. The graphic material is presented in annex A. The layout of the site, including the connection of components, is presented in annex B.

List of keywords: intelligent system, computer vision, convolutional neural networks, YOLOv11, Flask, PostgreSQL, web application, disease diagnostics, agricultural plants, tomatoes, database, module, component, user interface, machine learning, image processing, Python, Google Colab, PyTorch, information technology.

The object of development is an intelligent system for diagnosing diseases of agricultural plants, in particular tomatoes, using computer vision and machine learning technologies.

The goal of the final qualifying work is to improve the efficiency of diagnosing diseases of agricultural plants by creating an intelligent system that allows you to upload plant images, identify diseases and provide recommendations for their treatment and prevention.

In the process of creating the system, the main entities were identified by designing a database, modules and classes were used to process images and interact with the database, a user interface based on HTML / CSS and the Flask framework was developed, and a YOLOv11 machine learning model was implemented to analyze plant images and identify diseases.

The system was developed using the Flask framework for the server side, PostgreSQL for storing disease and recommendation data, and Python and PyTorch for implementing and training the machine learning model.

The developed system was successfully tested.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	11
1 Анализ предметной области	13
1.1 Описание предметной области	13
1.2 Роль искусственного интеллекта в сельском хозяйстве	14
1.3 Сверточные нейронные сети	15
1.4 Анализ основных видов заболеваний томатов	18
1.5 Анализ существующих решений	28
2 Техническое задание	32
2.1 Основание для разработки	32
2.2 Цель и назначение разработки	32
2.3 Актуальность темы разработки	33
2.4 Этапы разработки	33
2.5 Требования к программной системе	34
2.5.1 Требования к данным	34
2.5.2 Функциональные требования	34
2.5.3 Сценарий прецедента пользователя «Загрузка изображения»	35
2.5.4 Сценарий прецедента пользователя «Просмотр описания проблемы»	35
2.5.5 Сценарий прецедента пользователя «Просмотр рекомендаций по лечению и профилактике»	36
2.5.6 Сценарий прецедента пользователя «Просмотр рекомендаций по профилактике»	36
2.6 Требования пользователя к интерфейсу web-интерфейса	36
2.7 Нефункциональные требования	37
2.7.1 Требования к программному обеспечению	37
2.8 Ограничения	38
2.8.1 Требования к аппаратному обеспечению	38
2.9 Требования к оформлению документации	38
3 Технический проект	39

3.1	Общая характеристика организации решения задачи	39
3.2	Обоснование выбора технологии проектирования	39
3.2.1	Язык программирования Python	39
3.2.2	Google Colab	39
3.2.3	Фреймворк PyTorch	40
3.2.4	YOLO 11	40
3.2.5	Фреймворк Flask	41
3.2.6	PostgreSQL	42
3.3	Архитектура программной системы	42
3.3.1	Архитектура нейронной сети	43
3.3.1.1	Backbone	46
3.3.1.2	Neck	47
3.3.1.3	Head	48
3.4	Структура модулей программы	48
3.5	Структура базы данных	49
4	Рабочий проект	52
4.1	Спецификация системы	52
4.1.1	Модуль databaseManager	52
4.1.2	Модуль imageProcessor	53
4.1.3	Модуль model	53
4.1.4	Модуль trainModel	54
4.1.5	Модуль app	55
4.2	Обучения модели	56
4.2.1	Описание датасета	56
4.2.2	Результаты обучения	56
4.3	Тестирование модели	59
4.3.1	Анализ F1-меры в зависимости от порога уверенности	59
4.3.2	PR-кривые: точность и полнота по классам	60
4.3.3	Нормализованная матрица ошибок	61
4.4	Тестирование интеллектуальной системы	64
	ЗАКЛЮЧЕНИЕ	73

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	73
ПРИЛОЖЕНИЕ А Представление графического материала	80
ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы	94
На отдельных листах (CD-RW в прикрепленном конверте)	102
Сведения о ВКРБ (Графический материал / Сведения о ВКРБ.png)	Лист 1
Цель и задачи разработки (Графический материал / Цель и задачи разработки.png)	Лист 2
Диаграмма прецедентов (Графический материал / Диаграмма прецедентов.png)	Лист 3
Диаграмма компонентов (Графический материал / Диаграмма компонентов.png)	Лист 4
Архитектура YOLO 11 (Графический материал / Архитектура YOLO 11.png)	Лист 5
Структура модулей (Графический материал / Структура модулей.png)	Лист 6
ER-диаграмма (Графический материал / ER-диаграмма.png)	Лист 7
Динамика метрик обучения (Графический материал / Динамика метрик обучения.png)	Лист 8
Кривые F1-меры в зависимости от порога уверенности (Графический материал / Кривые F1-меры в зависимости от порога уверенности.png)	Лист 9
Кривые точности и полноты (Графический материал / Кривые точности и полноты.png)	Лист 10
Нормализованная матрица ошибок (Графический материал / Нормализованная матрица ошибок.png)	Лист 11
Результат анализа фотографии (Графический материал / Результат анализа фотографии.png)	Лист 12
Заключение (Графический материал / Заключение.png)	Лист 13

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

БД – база данных.

ИС – информационная система.

ИТ – информационные технологии.

ПО – программное обеспечение.

РП – рабочий проект.

СУБД – система управления базами данных.

ТЗ – техническое задание.

ТП – технический проект.

CNN – свёрточная нейронная сеть.

ИИ – искусственный интеллект.

UML (Unified Modelling Language) – язык графического описания для объектного моделирования в области разработки программного обеспечения.

ВВЕДЕНИЕ

Современное сельское хозяйство находится на пороге трансформации. Это обусловлено стремительным развитием цифровых технологий. Повышение продуктивности и качества аграрного производства требует внедрения инновационных решений, способных минимизировать потери и оптимизировать процессы. Одной из ключевых задач остается раннее выявление заболеваний сельскохозяйственных культур, таких как томаты, чья восприимчивость к патологиям существенно влияет на урожайность и экономическую устойчивость хозяйств [8].

Традиционные методы диагностики, основанные на визуальном осмотре или лабораторных исследованиях, часто оказываются трудоемкими и медленными, а также требуют привлечения квалифицированных специалистов. Но уже на сегодняшний день, технологии искусственного интеллекта (ИИ) и компьютерного зрения представляют новые способы автоматизацию процессов мониторинга здоровья растений, которые смогут обеспечить высокую точность и оперативность [9].

Создание удобных инструментов на базе машинного обучения открывает новые возможности в сельском хозяйстве. Такие решения позволят эффективно анализировать визуальные данные, выявлять признаки заболеваний растений и предлагать обоснованные рекомендации по их лечению и профилактике. Интуитивно понятный интерфейс и автоматизация процессов делают данный подход доступным для широкого круга пользователей.

Цель данной работы – разработка интеллектуальной системы для выявления заболеваний сельскохозяйственных растений, в частности томатов, на основе технологий машинного обучения и веб-технологий. Для достижения этой цели необходимо решить следующие задачи:

- определить требования к системе диагностики и описать её функциональные возможности;
- разработать архитектуру программного комплекса;
- обучить свёрточную нейронную сеть распознавать заболевания;

- реализовать веб-приложение для загрузки изображений растений, анализа заболеваний и предоставления рекомендаций;

- провести комплексное тестирование системы для оценки её надежности и точности.

Структура и объем работы. Отчет состоит из введения, 4 разделов основной части, заключения, списка использованных источников, 2 приложений. Текст выпускной квалификационной работы равен 102 страницам.

Во введении сформулирована цель работы, поставлены задачи разработки, описана структура работы, приведено краткое содержание каждого из разделов.

В первом разделе проводится анализ предметной области, включая сбор информации о болезнях томатов и методах их диагностики с использованием искусственного интеллекта.

Во втором разделе описываются требования к разрабатываемой системе диагностики заболеваний.

В третьем разделе представлены проектные решения для интеллектуальной системы, архитектура модели машинного обучения и веб-приложения.

В четвертом разделе приведены программные компоненты системы, их описание и результаты тестирования.

В заключении излагаются основные результаты работы, полученные в ходе разработки.

В приложении А представлен графический материал.

В приложении Б представлены фрагменты исходного кода.

1 Анализ предметной области

1.1 Описание предметной области

В настоящее время, когда население планеты продолжает расти, сельское хозяйство сталкивается с очень важной задачей – обеспечение продовольственной безопасности. Поэтому поддержание здоровья сельскохозяйственных растений напрямую влияет на объемы и стабильность урожая [5].

На сегодняшний день, проблема защиты растений от болезней и вредителей по-прежнему актуальна. По данным ФАО, по состоянию на 2024 год, потери сельскохозяйственных культур от вредителей и болезней растений достигают 40%. В России ситуация также остается непростой: до 25% зерновых культур гибнет из-за ржавчины, мучнистой росы, фузариоза и других заболеваний. Потери урожая картофеля из-за фитофтороза, альтернариоза и вирусных инфекций составляют 20–30%. Рис страдает от пирикулярноза, подсолнечник — от гнилей, сахарная свёкла — от церкоспороза, а яблони и сливы — от парши и монилиоза [1]. Болезни растений, вызываемые разнообразными патогенами и неблагоприятными условиями, представляют собой серьезную угрозу для сельского хозяйства. Поэтому важно разработать эффективные методы их диагностики.

Традиционные методы для выявления болезней включают в себя несколько подходов:

1. Визуальный осмотр: этот метод опирается на опыт и знания квалифицированных специалистов, предполагает анализ видимых симптомов, которые видны на различных частях растения [6].

2. Микроскопический анализ: данная методика позволяет найти патогены в тканях растения. Для него используются оптические микроскопы, что помогает более точно определить источник заболевания [2].

3. Молекулярные методы: этот метод исследования позволяет выявить генетические и молекулярные изменения в организме. Это обеспечивает высокую точность и специфичность диагностики [11].

Однако, традиционные методы диагностики имеют свои ограничения. Они трудоемки, требуют использования специализированного оборудования и привлечения высококвалифицированных специалистов. Кроме того, их недостатками является то, что они могут потребовать много времени. Поэтому использование методов искусственного интеллекта и глубокого обучения может повысить точность и эффективность диагностики заболеваний сельскохозяйственных культур. [52]

1.2 Роль искусственного интеллекта в сельском хозяйстве

Искусственный интеллект находит всё более широкое применение в аграрном секторе. Его внедрение способствует снижению зависимости от ручного труда, оптимизации производственных процессов и повышению устойчивости агропроизводства перед лицом климатических и биотических рисков.

Одним из направлений применения ИИ является мониторинг состояния посевов. С использованием спутниковых и беспилотных летательных аппаратов, которые оснащены высокоточным оптическим и мультиспектральным оборудованием, осуществляется регулярная съёмка посевов. Затем полученные изображения анализируются с помощью алгоритмов машинного и глубокого обучения для выявления отклонений в цвете, текстуре и структуре растений. Такой подход позволяет оперативно локализовать проблемные участки и сократить использование химических препаратов за счёт точечного воздействия [53].

Ещё одно важное направление – моделирование урожайности с помощью предсказания. На основе данных о метеоусловиях, составе почв, предыдущем урожае и многих других факторах формируются модели, способные предсказывать какой объём урожая может быть при текущих условиях. Эти системы активно применяются в странах ЕС, США, Китае и Индии для планирования сельскохозяйственных работ и оценки экономических рисков [52].

Третье направление – это диагностика заболеваний растений с использованием мобильных и веб-приложений. Эти платформы используют обу-

ченные сверточные нейронные сети, способные по фото растения сделать предположение о наличии конкретных заболеваний. Обучение таких моделей осуществляется на десятках тысяч размеченных изображений растений, которые пострадали от болезней или вредителей. Точность идентификации может превышать 90%, что делает технологию полезной для мелких фермеров, не имеющих доступа к агрономам [54].

Кроме того, активно развиваются роботизированные системы на базе ИИ, способные выполнять операции по уходу за растениям. Так, компании Naïo Technologies из Франции, Ecorobotix из Швейцарии, Small Robot Company из Великобритании разрабатывают автономных агроботов, которые смогут облегчить работу фермерам, путём автоматизации сложных и требующих высокой точности задач [4].

1.3 Сверточные нейронные сети

Сверточные нейронные сети (CNN, англ. Convolutional Neural Networks) представляют собой тип глубоких нейронных сетей, которые специально разработаны для обработки данных с сеточной структурой, таких как двумерные изображения. CNN кардинально изменили подход к анализу изображений, благодаря способности автоматически извлекать релевантные признаки из данных. Это сделало возможным решение таких задач, как классификация, сегментация, обнаружение и распознавание объектов с высокой точностью и устойчивостью к шумам и искажениям.

Их основной принцип работы заключается в применении свёрточных фильтров, которые последовательно выявляют локальные закономерности в изображениях, начиная с простых признаков, таких как границы, и заканчивая высокоуровневыми абстракциями, такими как формы, контуры, объекты [22].

Одной из первых архитектур, продемонстрировавших эффективность CNN, стала LeNet-5, разработанная Яном Лекуном в 1998 году для распознавания рукописных цифр. Она включала в себя два сверточных и два пулинг слоя, а также полносвязные слои на выходе. Несмотря на свою простоту,

ту, LeNet-5 продемонстрировала фундаментальные принципы, которые были заложены в более поздние архитектуры. На рисунке 1.1 представлена архитектура LeNet-5.

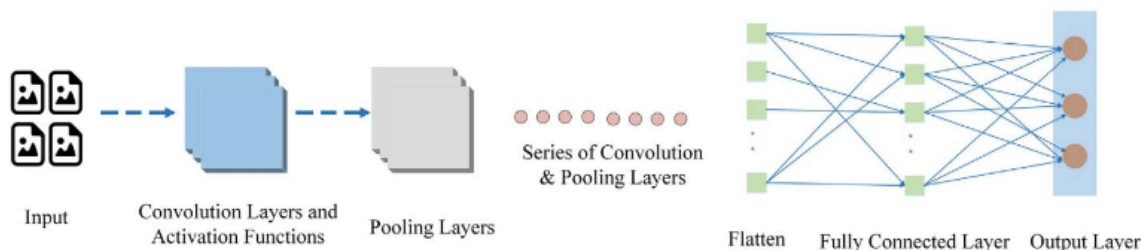


Рисунок 1.1 – Архитектура LeNet-5

Существенный прорыв в развитии сверточных сетей произошел, когда была разработана AlexNet в 2012 году. Она выиграла соревнование ImageNet, сократив ошибку классификации почти вдвое по сравнению с предыдущими моделями. AlexNet глубже использовала сверточные архитектуры, функцию активации ReLU, нормализацию по пакету и технику «дропаут» для уменьшения переобучения. Эта модель показала эффективность свёрточных сетей и стала катализатором их развития в целом. На рисунке 1.2 представлена архитектура AlexNet.

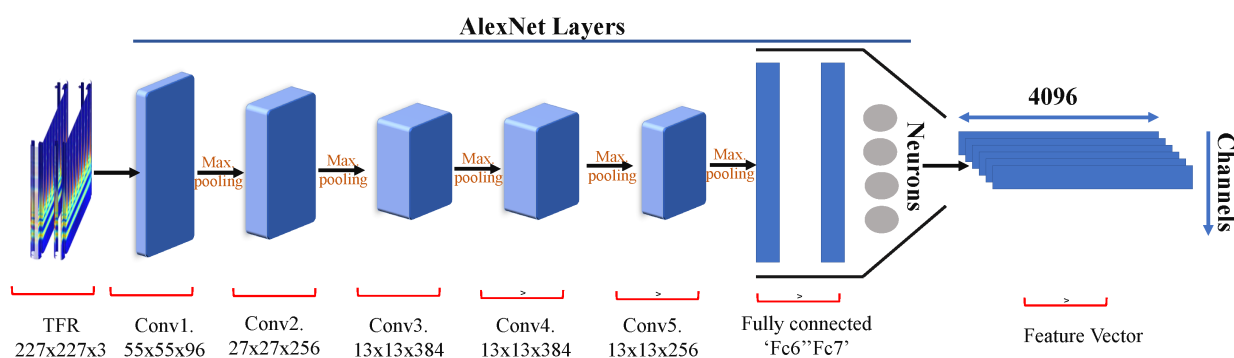


Рисунок 1.2 – Архитектура AlexNet

Впоследствии были разработаны более глубокие и эффективные архитектуры, такие как VGG, в которой использовались только 3×3 свёрток и 2×2 пуллинга, ResNet, в которой введены остаточные связи для облегчения обучения очень глубоких сетей, и GoogLeNet, которая использовала модули Inception. Эти модели стали стандартом в компьютерном зрении. Они приме-

няются в различных отраслях: от медицины и промышленной диагностики до систем анализа спутниковых снимков.

Развитие CNN позволило перейти от задач классификации к задачам более высокого уровня сложности, в частности к сегментации. Суть этой задачи заключается в построении пиксельных масок, которые соответствуют различным объектам или классам на изображении. Такой подход позволил расширить функциональность моделей по сравнению с обычной классификацией. Примерами соответствующих архитектур являются U-Net, DeepLab и Mask R-CNN, каждая из которых использует комбинацию сверточных и деконволюционных операций, а также механизмы пропуска слоев и многоуровневого объединения признаков [23] .

Одним из ключевых направлений в развитии методов компьютерного зрения стало обнаружение объектов. Данная задача предполагает совместное решение задач классификации и локализации. Модель при этом должна не только определить принадлежность объекта к определённому классу, но и задать его пространственные координаты на изображении.

Для её решения были разработаны архитектуры, сочетающие высокую производительность и точность. Наиболее известными моделями являются YOLO (You Only Look Once), SSD (Single Shot Multibox Detector) и Faster R-CNN.

Первая из них представляет собой одноэтапную архитектуру. Она осуществляет предсказание координат и классов объектов в рамках единой свёрточной сети. Благодаря своей архитектуре, отличается высокой вычислительной эффективностью и возможностью обработки изображений в режиме, близком к реальному времени.

Модель SSD тоже основана на одноэтапном подходе. Она использует многомасштабные карты признаков для распознавания объектов различных размеров, что делает её более универсальной при работе с разнородными данными.

Faster R-CNN имеет двухэтапную архитектуру. Первый модуль генерирует предложений о потенциальной области объектов, а второй выполняет

классификацию. Такой подход обеспечивает высокую точность за счёт явного разделения этапов локализации и распознавания.

1.4 Анализ основных видов заболеваний томатов

Томаты (*Solanum lycopersicum*) – это одна из ключевых овощных культур, которую выращивают во всём мире. Она широко используется при приготовлении пищи и обладает высокой агрономической значимостью. В 2023 году мировое производство томатов составило более 190 млн тонн [13]. Это показывает устойчивую тенденцию роста, даже на фоне климатических и биотических стрессов.

Однако томаты подвержены значительным потерям урожая из-за болезней и вредителей. Потери могут достигать 50% в условиях высокой заболеваемости, особенно при отсутствии своевременного выявления заболеваний [15].

Одной из таких болезней является альтернариоз, или сухая пятнистость листьев. Это грибковое заболевание, вызываемое патогенами рода *Alternaria*, преимущественно *Alternaria solani* и *Alternaria tomatophila*. Заболевание поражает листья, стебли, плоды и приводит к значительным потерям урожая.

Первые признаки болезни проявляются на нижних листьях в виде небольших тёмно-коричневых пятен с концентрическими кольцами. Со временем пятна увеличиваются, сливаются и охватывают большую часть листовой поверхности, вызывая преждевременное отмирание листьев. На стеблях и черешках появляются продольные тёмные полосы, которые могут приводить к их растрескиванию. На плодах появляются пятна гнили болотного цвета.

Развитию болезни способствуют: высокая влажность воздуха, температура в диапазоне от +20°C до +30°C, густая посадка, мешающая циркуляции воздуха. Споры гриба могут сохраняться в почве и на растительных остатках до 2–3 лет, поэтому необходимо соблюдать севооборот и тщательно убирать поля после сбора урожая [2]. На рисунке 1.3 представлены признаки болезни на листьях.



Рисунок 1.3 – Альтернариоз

Одним из наиболее опасных заболеваний томатов является фитофтороз, вызываемый оомицетом *Phytophthora infestans*. Эта патогенная микроскопическая структура также поражает картофель и другие растения семейства паслёновых. Фитофтороз способен стремительно распространяться в условиях высокой влажности и умеренных температур, вызывая массовое заражение посадок и значительное снижение урожайности.

Первые признаки заболевания появляются на нижних листьях в виде водянистых пятен бурого или серо-зелёного цвета. По мере развития болезни пятна становятся более выраженными, часть листа вокруг них отмирает, и образуется светлый ободок. С нижней стороны листьев может наблюдаться белый налёт, представляющий собой мицелий патогена. Поражение распространяется вверх по растению. Оно охватывает стебли и плоды. На плодах фитофтороз вызывает бурые, твёрдые пятна, часто с чёрной каймой. Такие плоды быстро загнивают, особенно в условиях хранения [3]. На рисунке 1.4 представлены признаки болезни на листьях.

Кладоспориоз томатов, также известный как бурая пятнистость, — это грибное заболевание. Патоген поражает преимущественно листья томатов,



Рисунок 1.4 – Фитофтороз

реже – стебли и плодоножки. У болеющего томата значительно снижается урожайность и качество самих плодов.

Болезнь вызывается грибом *Cladosporium fulvum*, который развивается на межклеточном уровне и продуцирует токсины, нарушающие физиологические процессы в растении. Грибки распространяются спорами, которые разносятся с потоками воздуха, а также через инструменты, одежду или капли воды. Также источником инфекции могут быть заражённые растительные остатки или семена, не прошедшие предварительную обработку [12].

Наиболее характерные признаки кладоспориоза проявляются на нижней стороне листьев в виде светло-зелёных, жёлтых, а затем бурых пятен округлой или неправильной формы. На рисунке 1.5 представлено больное растение. Поражённые листья постепенно скручиваются, усыхают и опадают.



Рисунок 1.5 – Кладоспориоз

Септориоз томатов, или белая пятнистость, – это одно из наиболее распространённых грибных заболеваний, вызываемое патогеном *Septoria lycopersici* Speg. Болезнь поражает преимущественно листву в фазу активного роста растений.

По данным специалистов Россельхозцентра, септориоз регистрируется ежегодно во всех регионах России, но чаще всего в зонах с повышенной влажностью [8].

Первые признаки проявляются на нижних листьях в виде маленьких светло-серых пятен с тёмной каймой. Постепенно пятна увеличиваются до 2–5 мм в диаметре, становятся округлыми, с отчетливо выраженной тёмной границей. В центре пятен можно разглядеть мелкие тёмные точки — пикниды гриба. Изображение больного листа представлено на рисунке 1.6.



Рисунок 1.6 – Септориоз томата

При сильном развитии болезни пятна сливаются, листовая пластинка желтеет, высыхает и опадает. Это нарушает фотосинтез и ослабляет растение, особенно в период формирования завязей.

Мозаичный вирус томатов – одно из наиболее известных и широко распространённых вирусных заболеваний томатов. Болезнь отличается высокой заразностью и может вызвать значительное снижение урожайности и ухудшение качества плодов.

Вирус впервые был описан в начале XX века. Возбудитель относится к роду *Tobamovirus* и представляет собой устойчивую РНК-вирусную частицу [20].

Первые признаки заражения чаще всего появляются на молодых листьях. Они приобретают характерную мозаичную окраску, при которой чередуются светло-зелёные и тёмно-зелёные участки. На стеблях и черешках

появляются бурые полосы, а плоды теряют равномерность окраски и могут быть покрыты кольцевыми пятнами. В условиях высокой температуры и интенсивного солнечного света симптомы становятся более выраженными. Изображение зараженного томата представлено на рисунке 1.7.



Рисунок 1.7 – Мозаичный вирус

Передача вируса происходит через контакт с руками, одеждой, садовым инвентарём, на которых есть вирус, или при контакте с насекомыми-переносчиками, такими как тля и табачный трипс [5].

Вирус жёлтой курчавости листьев томата – ещё одно опасное вирусных заболеваний. Он относится к роду *Begomovirus*, семейству *Geminiviridae*, и характеризуется способностью к быстрому распространению, особенно в регионах с тёплым климатом.

На территории России заболевание наиболее часто встречается в южных регионах — Краснодарском крае, Ростовской области, Крыму и Ставро-

поле, где благоприятные климатические условия способствуют активному размножению основного переносчика – белокрылки (*Bemisia tabaci*). Она является единственным распространителем данного вируса [14]. Изображение зараженного томата представлено на рисунке 1.8.



Рисунок 1.8 – Вирус жёлтой курчавости

У заражённых растений листья уменьшаются в размере, становятся ломкими, закручиваются вверх и приобретают лимонно-жёлтую окраску. Снижается способность к фотосинтезу, и, как следствие, ограничивается формирование цветков и завязей, что приводит к недостаточному развитию плодов или полному отсутствию урожая[15].

Минирующая мушка – один из опаснейших вредителей овощных культур. Они наносят серьёзный вред растениям, как на стадии рассады, так и в период плодоношения. Эти мелкие двукрылые насекомые относятся к семейству *Agromyzidae* и отличаются высокой плодовитостью и быстрой адапта-

цией к инсектицидам. На изображении 1.9 представлена минирующая мушка [16].



Рисунок 1.9 – Минирующая мушка

Самки прокалывают эпидермис листьев для откладки яиц. Вылупившиеся личинки начинают проделывать ходы в мезофилле листа, и, тем самым, разрушают фотосинтетическую ткань. Визуально повреждения проявляются в виде извилистых светлых линий – мин, которые увеличиваются в размерах по мере роста личинки. Повреждения представлены на изображении 1.10. При высокой численности вредителя может быть снижение урожайности до 40% [ссылка]ФГБУ «Россельхозцентр», 2021).

Мушки особенно опасны в теплицах, где температурно-влажностные условия способствуют быстрому размножению. В одном поколении может развиваться до 600–700 яиц, а полный цикл развития при благоприятных условиях занимает 15–20 дней.



Рисунок 1.10 – ”Мины”

Паутинный клещ – один из наиболее опасных вредителей, поражающих более 200 видов растений, включая томаты. Его биология, малые размеры и высокая устойчивость к пестицидам делают борьбу с ним крайне сложной [ссылка]. Особь клеща представлена на изображении 1.11.

Развитие клеща проходит особенно интенсивно при температуре 25–35 °С и относительной влажности менее 60%. Полный цикл развития — от яйца до взрослой особи — может занимать всего 7–10 суток, что позволяет вредителю развивать до 20 поколений в год в теплицах. Самки откладывают до 100–150 яиц за жизнь, формируя плотные колонии в межжилковых пространствах листьев [19].

На томатах паутинный клещ вредит в основном с нижней стороны листьев, прокалывая эпидермис и высасывая клеточный сок. Первичные симптомы проявляются в виде мелких светлых точек – хлорозов, которые со временем сливаются и придают листьям мраморный вид. При сильной степени заражения листья буреют, скручиваются, засыхают и опадают. Также можно наблюдать тонкую паутину, покрывающую нижнюю сторону листьев, черешки и даже соцветия. Паутина служит защитой для колоний клещей и яиц



Рисунок 1.11 – Паутинный клещ

от внешних воздействий и инсектицидов. На изображении 1.12 представлен, томатный куст, пострадавший от клещей.



Рисунок 1.12 – Колония паутинных клещей на кусте томата

1.5 Анализ существующих решений

С помощью анализу существующих решений с похожей тематикой, можно выделить их достоинства и недостатки, чтобы использовать эту информацию при разработке своего решения. Были выбраны три веб-сервиса, в которых используются технологии компьютерного зрения и искусственного интеллекта:

- "plantix";
- "culiKure";
- "pdd.jinr".

Все указанные платформы решают задачи идентификации болезней растений по загруженным изображениям, однако имеют отличия в интерфейсе, используемых технологиях и функционале.

Так Plantix – это мобильное приложение, ориентированное на фермеров и агрономов. Пользователь делает фото растения, а приложение анализирует его и предоставляет подробную информацию о заболевании, методах лечения. Интерфейс рассчитан на массового пользователя и поддерживает множество региональных языков. Однако его минусом является то, что это мобильное приложение, поэтому нет возможности использовать его через браузер. На изображении 1.13 представлен скриншот Plantix.

CultiKure – это веб-приложение, предназначенное для помощи пользователям в выявлении и устранении заболеваний растений. Оно анализирует изображения листьев с помощью модели VGG16 и выдаёт результаты с информацией по болезням. Само приложение построено на базе фреймворка Flask. Минусом данного решения является то, что оно использует не самую лучшую версию модели, и, как результат, может выдавать не самые точные предсказания [48]. На изображении 1.14 представлен скриншот CultiKure.

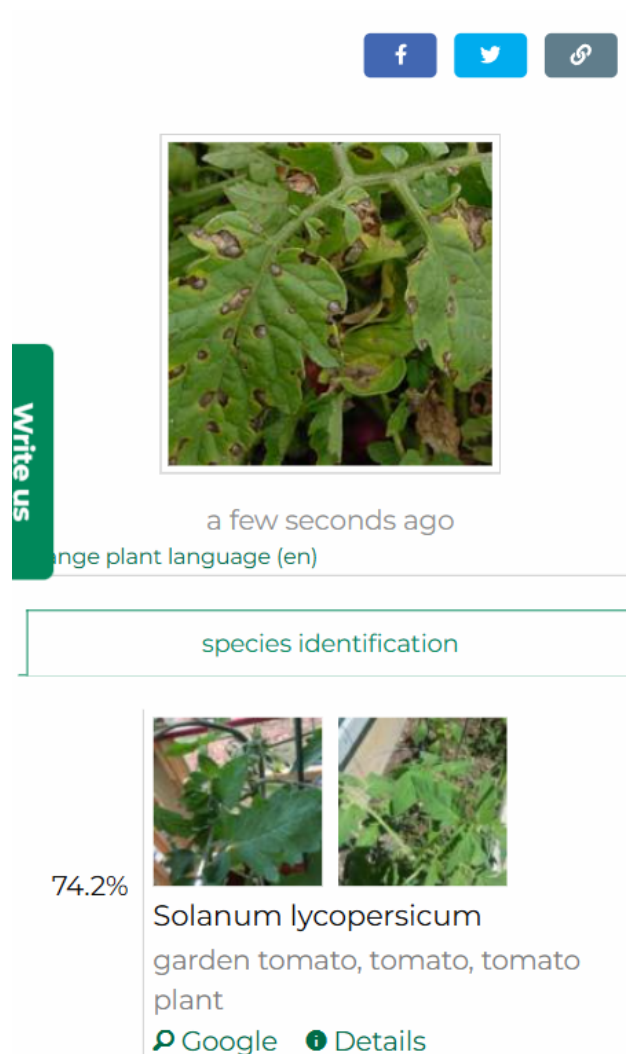
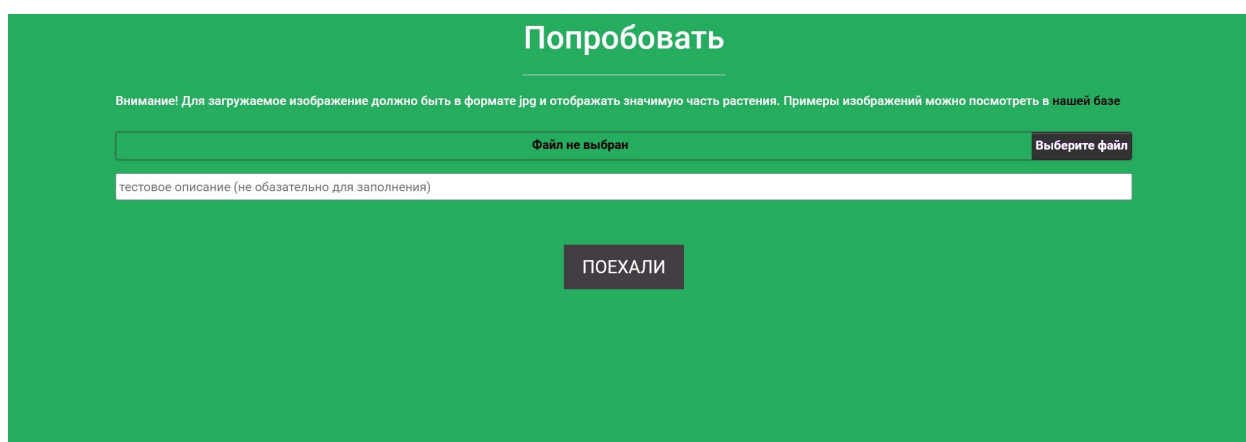


Рисунок 1.13 – Интерфейс Plantix



Рисунок 1.14 – Интерфейс Cultikure

PDD.JINR.RU является исследовательским проектом, созданным в ”Объединённом институте ядерных исследований”. Веб-интерфейс позволяет загружать изображения листьев и определять заболевание с помощью сверточной нейросети. Акцент данного решения делается на точность и демонстрацию работы алгоритма ИИ. Однако визуальное оформление сайта значительно уступает в удобстве и современности свои конкурентам. На изображении 1.15 представлен интерфейс, а на 1.16 - результат предсказания PDD.JINR.RU.



The screenshot displays the web interface of PDD.JINR.RU, which has a green background. At the top, the word "Попробовать" (Try) is centered in white. Below it, a small line of text reads: "Внимание! Для загружаемое изображение должно быть в формате jpg и отображать значимую часть растения. Примеры изображений можно посмотреть в нашей базе" (Attention! For the uploaded image must be in jpg format and display a significant part of the plant. Examples of images can be seen in our database). The main form consists of a large green rectangular area with a white border. Inside this area, there is a text input field with the placeholder "Файл не выбран" (File not selected) and a button labeled "Выберите файл" (Choose file). Below the input field is a smaller white text input field with the placeholder "тестовое описание (не обязательно для заполнения)" (test description (not mandatory for filling)). At the bottom center of the green area is a dark green button with the text "ПОЕХАЛИ" (GO).

Рисунок 1.15 – Интерфейс PDD.JINR.RU

Вероятнее всего это: Аскохитоз



Возбудители аскохитоза: грибы, принадлежащие к роду *Ascochyta*. Аскохитоз является опасным заболеванием, поражающим тыкву, дыни, арбуз, горох, фасоль, свеклу, огурцы, смородину, крыжовник и некоторые другие культуры. Растения поражаются на протяжении всего периода вегетации. Основными симптомами заболевания является появление пятен различной окраски и формы, но всегда с бурым окаймлением. Отличительной чертой аскохитоза является обязательное образование пикнид - полых вместилищ, которые состоят из сплетения мицелия. В пикнидах формируются пикноспоры, которые разносят заболевание.

Пораженные аскохитозом семена приводят к снижению всхожести растений. У заболевших аскохитозом взрослых растений снижается урожайность. Недобор урожая может составлять до 50%. Плоды образуются недоразвитыми, плохого качества.

Заболеванию подвергаются все части растений. Размножению грибной культуры способствует повышенная влажность при выпадении обильных осадков в сочетании с температурой около +20-25°C. Источниками заражения являются пораженные аскохитозом семена или неубранные остатки предыдущего урожая. Когда преобладает чередование сухой и влажной погоды развитие заболевания замедляется, а при повышении температуры до +30°C и выше - заболевание прекращается.

Лечение аскохитоза заключается в избавлении от зараженных растений, сокращении объема полива. Также необходимо контролировать температурный режим для культур, выращиваемых в тепличных условиях, проводить обработку специальными фунгицидами, применять стимуляторы роста, укрепляющие иммунитет, проводить опудривание или обмазывание пораженных участков растворами медного купороса и мела, толченым углем.

Профилактические меры, предупреждающие возникновение аскохитоза, довольно обширны и незначительно разнятся в зависимости от конкретной культуры. Для предотвращения появления аскохитоза необходимо проводить своевременную очистку от растительных остатков предыдущих урожаев с посевных площадей и из теплиц, дезинфицирование используемой тары и садового инструмента, обеззараживание почвы, протравливание семенного материала перед посадкой. Также рекомендуется вносить в почву биопрепараты против аскохитоза, использовать регуляторы роста для повышения иммунитета растений и проводить регулярные опрыскивания фунгицидными препаратами, предотвращающими заражение.

Профилактика

Профилактические меры, предупреждающие возникновение аскохитоза огурца:

- Очистка теплиц от растительных остатков.
- Проведение дезинфицирования используемой тары и садового инструмента.
- Проведение обеззараживающей обработки почвы.
- Предотвращение переохлаждения растений.
- Тщательный контроль режим полива и проветривания в теплицах.
- Проведение обследования растений в зонах повышенного риска.
- Соблюдение севооборота.

Рисунок 1.16 – Интерфейс PDD.JINR.RU

2 Техническое задание

2.1 Основание для разработки

Полное наименование системы: «Разработка интеллектуальной системы для выявления заболеваний сельскохозяйственных растений». Основанием для разработки программы является приказ ректора ЮЗГУ от «04» апреля 2025 г. № 1696-с приказа «Об утверждении тем выпускных квалификационных работ и руководителей выпускных квалификационных работ».

2.2 Цель и назначение разработки

Программно-информационная система предназначена для диагностики заболеваний томатов по изображениям их листьев. Пользователями системы могут быть как агрономы, так и дачники.

Пользователи должны иметь возможность загружать фотографии поражённой листовой, получать результаты анализа и рекомендации по лечению и профилактике. Кроме того, система должна предоставлять краткое описание выявленного заболевания и возможные причины его возникновения.

Целью разработки является повышение эффективности сельскохозяйственного производства за счёт раннего выявления заболеваний и предоставления точных рекомендаций для их устранения. Это позволит сократить потери урожая, снизить затраты на химическую обработку и улучшить качество плодов.

Задачами данной разработки являются:

1. Создание базы данных для хранения информации о заболеваниях, их симптомах, способах лечения и профилактики.
2. Проектирование пользовательского интерфейса для удобной загрузки изображений и отображения результатов анализа.
3. Разработка и обучение свёрточной нейросети для точной классификации заболеваний по изображениям.

2.3 Актуальность темы разработки

Заболевания растений остаются одной из ключевых проблем в сельском хозяйстве. Они приводят к значительным потерям урожая и снижению его качества, поэтому крайне важной задачей является минимизация потерь за счёт внедрения современных технологий диагностики.

Разработка системы, основанной на использовании свёрточных нейронных сетей для распознавания заболеваний по фотографиям, актуальна по нескольким причинам. Во-первых, она позволяет сократить зависимость от квалификации специалистов, обеспечивая высокую точность диагностики даже для пользователей, которые не разбираются в агрономии. Во-вторых, такая система предоставляет возможность раннего выявления болезней, что существенно увеличивает шансы на успешное лечение. [54]

Кроме того, использование веб-технологий делает решение доступным для пользователей в любых регионах, где есть доступ к сети Интернет. Применение данного подхода может повысить эффективность сельскохозяйственного производства, сократить экономические потери и снизить экологическую нагрузку за счёт оптимизации использования химических средств защиты растений.

В сравнении с традиционными методами диагностики, предложенная система способна в течение нескольких секунд предоставить точный результат анализа, а воспользоваться ею можно везде, где есть Интернет. Эти факторы могут сделать данное решение удобным инструментом для современного сельского хозяйства.

2.4 Этапы разработки

Для реализации программной системы предполагается выполнение следующих этапов:

1. Изучение существующих решений, определение перечня заболеваний томатов и их визуальных симптомов.

2. Разработка структуры базы данных для хранения информации о заболеваниях, методах лечения и профилактики.
3. Сбор и разметка набора данных, обучение модели YOLO 11 и её тестирование.
4. Создание пользовательского интерфейса, интеграция CNN с веб-приложением.
5. Проведение функционального тестирования, проверка точности диагностики.

2.5 Требования к программной системе

2.5.1 Требования к данным

Входными данными для системы являются:

- изображения листьев растений, которые пользователь загружает для анализа.

Выходными данными для системы являются:

- информация о заболевании;
- данные об актуальных методах лечения и профилактики;
- сообщения об отсутствии информации о заболевании в базе.

2.5.2 Функциональные требования

Пользователю должны быть доступны следующие функции:

- загрузка фотографии пораженного листа томата для диагностики заболеваний;
- просмотр результатов анализа с информацией о выявленном заболевании;
- просмотр рекомендаций по лечению;
- просмотр информации по профилактике.

На рисунке 2.1 представлена диаграмма прецедентов для пользователя.

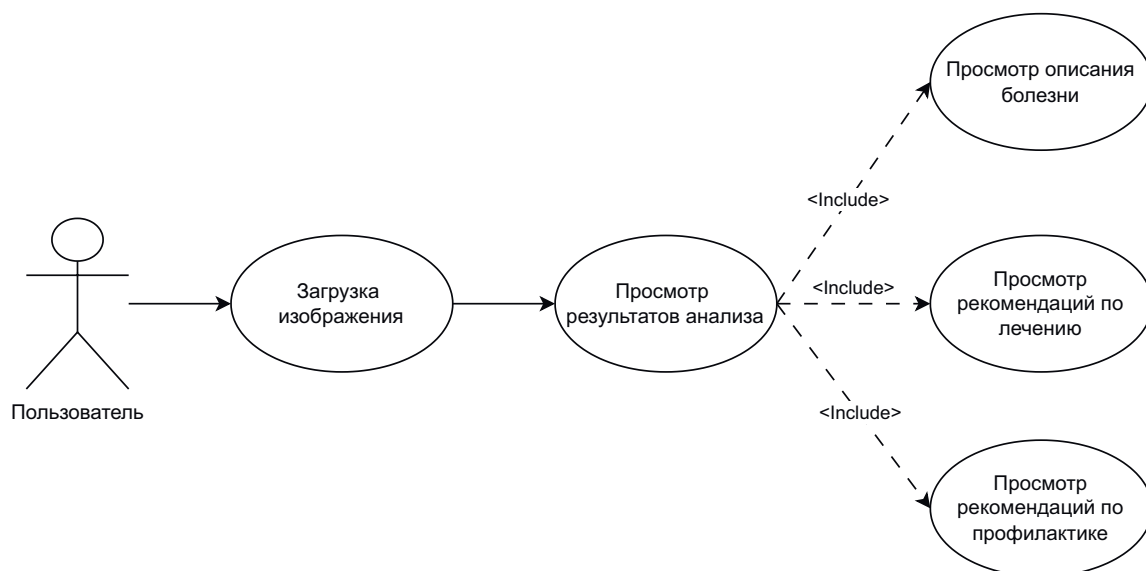


Рисунок 2.1 – Диаграмма прецедентов для пользователя

2.5.3 Сценарий прецедента пользователя «Загрузка изображения»

Основной успешный сценарий:

- пользователь заходит на главную страницу системы;
- выбирает опцию "Обзор...";
- выбирает изображение на своем устройстве;
- подтверждает выбор, нажав кнопку "Анализировать изображение";
- дожидается завершения анализа.

2.5.4 Сценарий прецедента пользователя «Просмотр описания проблемы»

Основной успешный сценарий:

- изображение проанализировано;
- пользователь переходит к параграфу "Описание";
- просматривает информацию о найденной проблеме.

2.5.5 Сценарий прецедента пользователя «Просмотр рекомендаций по лечению и профилактике»

Основной успешный сценарий:

- изображение проанализировано;
- пользователь переходит к параграфу ”Лечение”;
- просматривает способы устранить проблемы.

2.5.6 Сценарий прецедента пользователя «Просмотр рекомендаций по профилактике»

Основной успешный сценарий:

- изображение проанализировано;
- пользователь переходит к параграфу ”Профилактика”;
- просматривает методы профилактики болезни.

2.6 Требования пользователя к интерфейсу web-интерфейса

Интерфейс должен быть интуитивно понятным, чтобы агрономы и обычные пользователи могли легко взаимодействовать с системой. На рисунке 2.2 представлен макет страницы.

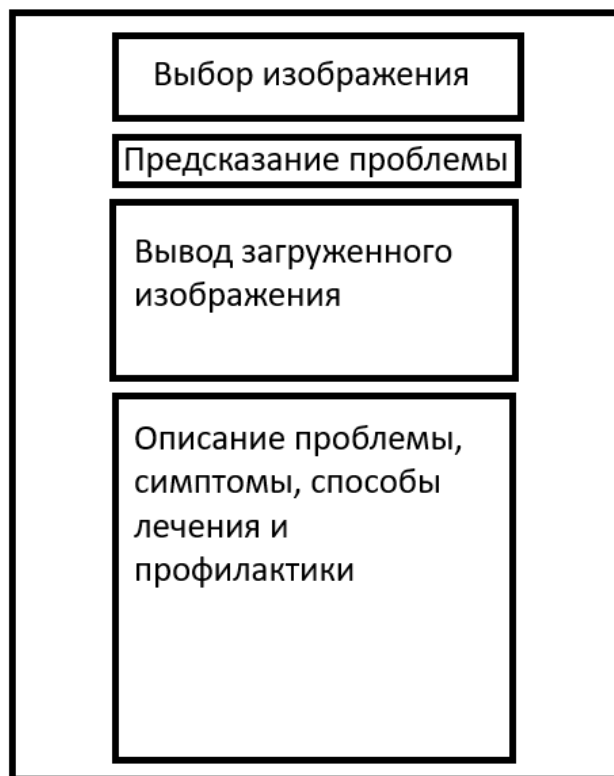


Рисунок 2.2 – Макет страницы

2.7 Нефункциональные требования

2.7.1 Требования к программному обеспечению

Для разработки и работы программной системы необходимы следующие компоненты:

- среда разработки Python для реализации и настройки алгоритмов;
- фреймворк Flask для создания веб-приложения;
- СУБД PostgreSQL для хранения данных о пользователях, результатах анализа и рекомендациях;
- инструменты машинного обучения Ultralytics и Pytorch для работы с CNN;
- библиотеки OpenCV, PIL для обработки изображений и Pandas, NumPy для работы с данными.

Предлагаемые технологии поддерживаются на всех популярных операционных системах, таких как Windows, macOS и Linux [40]. Выбор таких инструментов обусловлен их гибкостью, производительностью и большим

сообществом разработчиков, что упрощает разработку и последующую поддержку системы.

2.8 Ограничения

1. Система ориентирована на диагностику заболеваний томатов и не поддерживает другие культуры, поэтому может выдавать ошибочные предположения при загрузке изображений других культур.

2. Точность диагностики зависит от качества предоставленных входных данных.

3. Для работы системы требуется стабильное интернет-соединение.

2.8.1 Требования к аппаратному обеспечению

Для работы приложения требуется дисковое пространство не менее 2,5 Гб, минимум 2 Гб оперативной памяти и подключение к Интернету. Рекомендуется использовать процессор с 2 или более ядрами и частотой 2 ГГц или выше.

2.9 Требования к оформлению документации

Разработка программной документации и программного изделия должна производиться согласно ГОСТ 19.102-77 и ГОСТ 34.601-90. Единая система программной документации.

3 Технический проект

3.1 Общая характеристика организации решения задачи

Задача заключается в разработке интеллектуальной системы для диагностики заболеваний сельскохозяйственных растений, с фокусом на томатах. Система реализована как веб-приложение, которое анализирует изображения листьев, ищет признаки заболеваний с помощью нейросетевой модели YOLOv11 и предоставляет рекомендации по лечению и профилактике. Приложение поможет агрономам и дачникам быстро выявлять болезни растений и принимать меры для сохранения урожая.

3.2 Обоснование выбора технологии проектирования

Для создания приложения выбраны технологии, которые обеспечивают высокую производительность, удобство для пользователей и простоту интеграции с моделью машинного обучения.

3.2.1 Язык программирования Python

Python – высокоуровневый язык программирования с динамической типизацией. Его интерпретатор позволяет запускать код на различных платформах, включая Windows, macOS и Linux, без необходимости компиляции. Официальная документация поддерживает разные подходы к программированию, включая процедурный, объектно-ориентированный и функциональный стили, что даёт разработчикам гибкость [41] [42].

В 2025 году Python остаётся одним из самых популярных языков благодаря своему простому синтаксису и большому количеству библиотек, которые и позволяют создавать на нём веб-приложения и модели ИИ [43][40].

3.2.2 Google Colab

Google Colaboratory (или просто Colab) – это бесплатная облачная среда от компании Google для запуска Python-кода. Она позволяет работать с

блокнотами Jupyter прямо в браузере, не требуя установки дополнительных программ на компьютер пользователя [44].

Одним из главных преимуществ Colab является доступ к вычислительным ресурсам GPU и TPU, что делает его особенно полезным для обучения и тестирования моделей машинного обучения и нейросетей.

3.2.3 Фреймворк PyTorch

PyTorch – это инструмент, созданный для работы с нейронными сетями на Python. Он используется разработчиками, когда нужно быстро протестировать идею и собрать прототип, особенно в проектах, связанных с машинным обучением. Его ценят за гибкость, читаемость кода и большое количество готовых решений.

Открытый исходный код PyTorch позволяет любому разработчику внести изменения или адаптировать его под собственные задачи. Благодаря этому вокруг проекта сформировалось большое и активное сообщество. Постоянно появляются новые библиотеки, расширения и обучающие материалы, которые делают работу с фреймворком ещё проще [35].

Среди особенностей – поддержка как классических, так и продвинутых архитектур нейросетей. Встроенные модули позволяют обрабатывать данные, визуализировать результаты и отслеживать, как ведёт себя модель во время обучения.

Благодаря всему этому PyTorch активно применяется в разных сферах.

3.2.4 YOLO 11

YOLOv11 – это одна из последних разработок в серии алгоритмов для распознавания объектов на изображениях. Эту версию представила команда Ultralytics. Данная модель стала логичным продолжением предыдущих модификаций YOLO, сохранив при этом основной подход: находить и определять объекты за один проход по изображению [37].

Главное, за что ценят YOLO – это скорость и точность. Она справляется с обработкой видео и изображений практически в реальном времени.

Такая производительность достигается благодаря улучшенной архитектуре сверточной нейросети и доработанному механизму обучения. Разработчики также внедрили более точные методы предобработки данных, что заметно повысило стабильность работы модели [31].

На изображении 3.1 представлены результаты сравнения 11-ой версии с предшествующими ей.

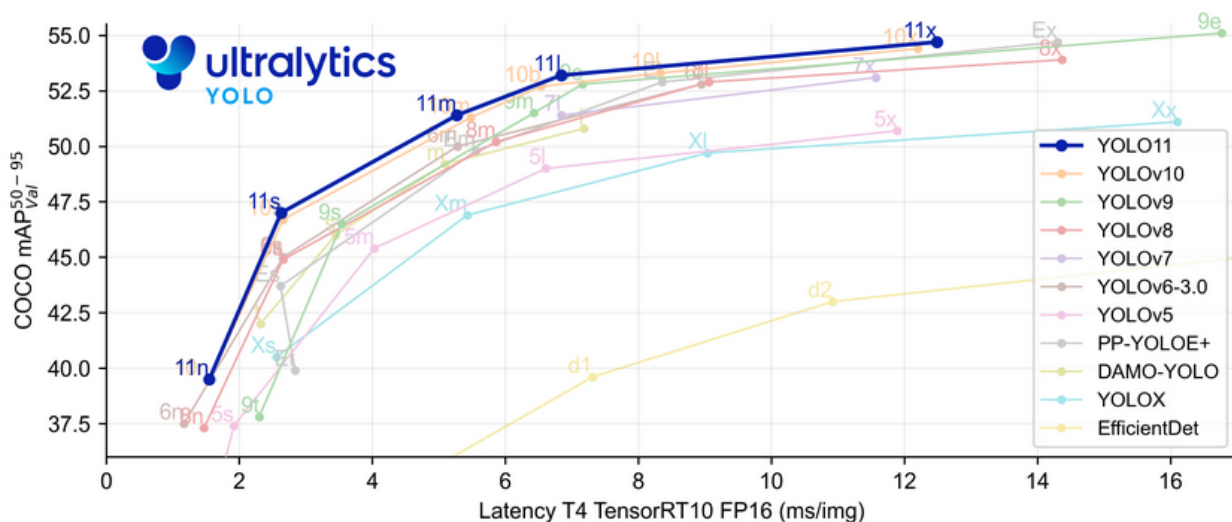


Рисунок 3.1 – График сравнения версий YOLO

3.2.5 Фреймворк Flask

Flask – это легковесный веб-фреймворк на языке Python, который часто используют для разработки простых и понятных веб-приложений. Он не навязывает структуру проекта, поэтому подойдет как новичкам, так и разработчикам, которым нужен контроль над архитектурой [45].

В отличие от более тяжелых решений, Flask не включает в себя избыточных модулей. Вместо этого он предлагает только базовые инструменты – маршрутизацию, работу с запросами и шаблонизатор [46].

Для разработки веб-интерфейса системы распознавания заболеваний растений Flask подойдет особенно хорошо. Он позволяет быстро реализовать обработку изображений, вывод результатов и взаимодействие с нейросетью. Благодаря простому синтаксису и большому количеству обучающей инфор-

мации, с реализацией приложений не возникает серьёзных трудностей, даже при ограниченном опыте в веб-программировании [49].

Фреймворк хорошо работает на любых операционных системах и легко разворачивается как на локальной машине, так и на сервере. Это делает Flask удобным выбором для небольших и средних проектов.

3.2.6 PostgreSQL

PostgreSQL – это СУБД с открытым исходным кодом. Она известна своей стабильностью, надёжностью и соответствием стандартам SQL [50].

PostgreSQL активно используется как в небольших проектах, так и в крупных корпоративных системах. Среди её ключевых особенностей – поддержка сложных запросов, транзакций, расширяемость и работа с JSON-данными. Это делает её удобной для современных веб-приложений [51].

В этой работе PostgreSQL используется для хранения данных о болезнях растений, их признаках и рекомендациях по лечению.

3.3 Архитектура программной системы

На рисунке 3.2 показана архитектура программной системы.

Диаграмма показывает, как устроено веб-приложение для диагностики заболеваний томатов. Она представлена в виде схемы с пятью основными блоками, соединёнными стрелками. Они указывают, как данные передаются между частями приложения. Каждая часть выполняет свою задачу, чтобы обеспечить работу системы.

Клиентский интерфейс – это веб-страница, через которую пользователь загружает изображение листьев. На этой же странице он видит результат анализа.

Веб-сервер стоит в центре схемы и управляет всем процессом. Он принимает запросы от интерфейса, направляет данные в нужные модули и возвращает результат пользователю.

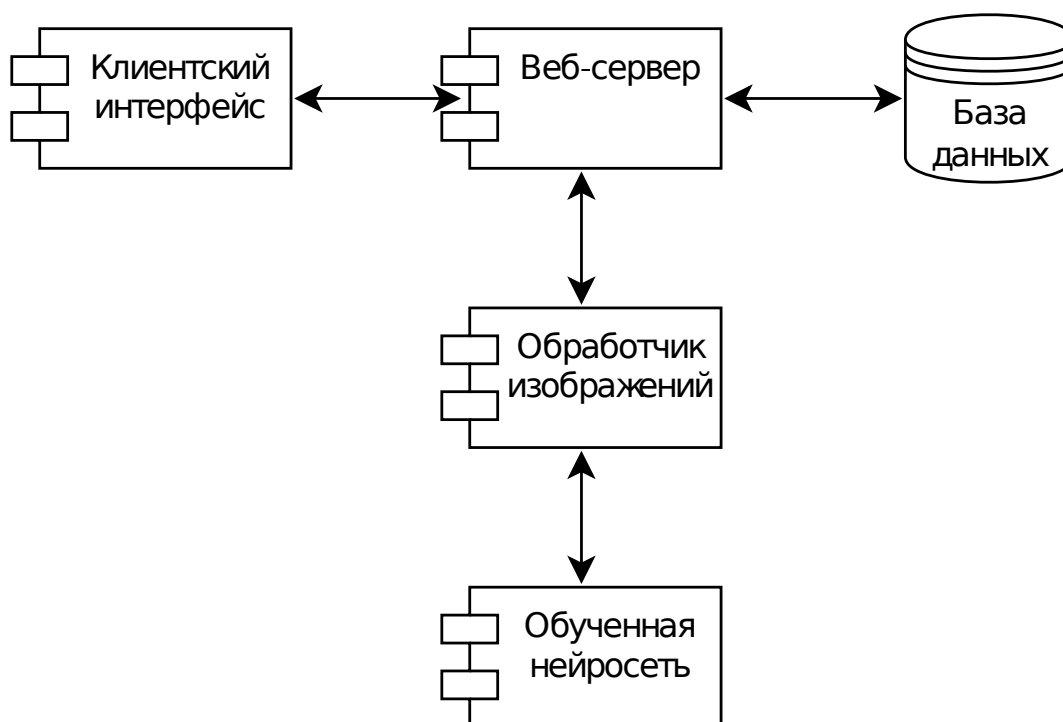


Рисунок 3.2 – Диаграмма компонентов

База данных хранит описание болезней томатов, методы лечения, способы профилактики. Сервер обращается к ней, если модель нашла недуг.

Модуль обработки изображений подготавливает загруженное фото для анализа. Он изменяет размер изображения, повышает резкость и контраст, а затем передаёт его в модель.

Нейронная сеть анализирует изображение и определяет, есть ли на фото болезнь и возвращает на сервер результат.

3.3.1 Архитектура нейронной сети

Основным параметром сети является nc , который указывает, сколько классов модель может распознавать. Для решаемой задачи устанавливаем nc равное 9.

Далее задаём варианты масштабирования, которые регулируют глубину, ширину сети и максимальное число каналов. Будем использовать глубину 0.50, ширину 0.25 и максимум 1024 канала. Данная модель состоит из 181 слоя и 2,624,080 параметров.

Архитектура YOLOv11 делится на три основные части: Backbone, Neck и Head. Они представлена на схеме 3.3.

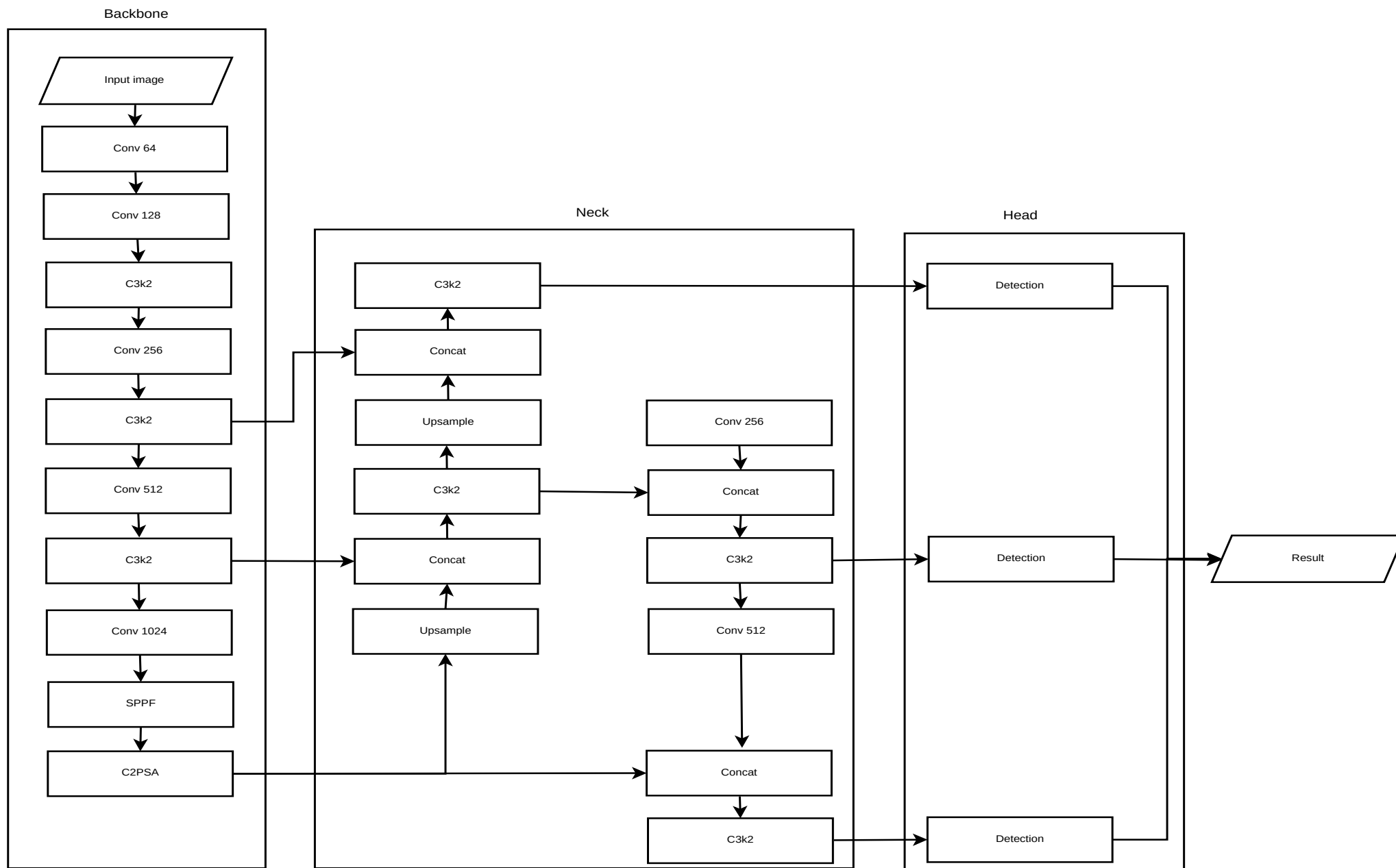


Рисунок 3.3 – Архитектура YOLO 11

3.3.1.1 Backbone

Backbone анализирует входное изображение размером 640x640 пикселей. Основной его задачей является выделение важных деталей, таких как края, текстуры или пятна на листе, которые характерны для заболеваний. Архитектура Backbone постепенно уменьшает размер изображения и увеличивает глубину признаков через следующие слои [37]:

1. Conv 64 – свёрточный слой с 64 фильтрами размером 3x3 и шагом 2, он уменьшает изображение с 640x640x3 до 320x320x64. При свёртке идёт обобщение крупных структур, таких как контуры листьев.

2. Conv 128 – свёрточный слой с 128 фильтрами размером 3x3 и шагом 2 уменьшает размер до 160x160x128, усиливая выделение текстурных особенностей.

3. C3k2 – улучшенный блок C3 с двумя свёрточными слоями и остаточными связями, которые пропускают часть входных данных через блок без изменений и добавляют их к выходу. Это предотвращает проблему затухающего градиента во время обратного распространения. На выходе имеем изображение 160x160x256.

4. Conv 256 уменьшает размер до 80x80x256, фокусируясь на признаках средней детализации, таких как пятна 10–20 пикселей.

5. Блок C3k2 с двумя свёрточными слоями преобразует данные до 80x80x512, выделяя более сложные текстуры, такие как пятна или повреждения листьев.

6. Слой conv 512 формирует карту размером 40x40x512, фокусируется на объектах среднего размера.

7. Блок C3k2 сохраняет размер 40x40x512, углубляет признаки.

8. Conv 1024 формирует карту размером 20x20x1024, выделяет крупные области поражения.

9. Блок C3k2 сохраняет размер 20x20x1024, углубляет признаки.

10. SPPF применяет пулинг с разными масштабами ядер (5x5, 9x9, 13x13), чтобы захватить признаки разной детализации. В отличие от дру-

гих методов пулинга, SPPF ускоряет этот процесс за счёт оптимизированных свёрток.

11. C2PSA основан на механизме само-внимания, который оценивает взаимосвязи между разными частями изображения. Это помогает модели сосредотачиваться на важных областях, таких как мелкие пятна или изменения цвета, игнорируя фоновые элементы.

Backbone формирует карты признаков $80 \times 80 \times 256$, $40 \times 40 \times 512$, $20 \times 20 \times 1024$ и передаёт в neck.

3.3.1.2 Neck

Neck собирает данные из backbone и готовит их к финальному этапу детекции. Эта часть объединяет информацию с разными масштабами, чтобы модель могла лучше находить мелкие объекты. Neck состоит из следующих модулей [32]:

1. Слой Upsample преобразует $20 \times 20 \times 1024$ в $40 \times 40 \times 1024$, используя интерполяцию, что улучшает анализ мелких деталей.
2. Слой Concat соединяет $40 \times 40 \times 1024$ с $40 \times 40 \times 512$, формируя $40 \times 40 \times 1536$. Это помогает сделать карту признаков полнее.
3. Блок C3k2 преобразует $40 \times 40 \times 1536$ в $40 \times 40 \times 512$, углубляет признаки.
4. Upsample преобразует $40 \times 40 \times 512$ в $80 \times 80 \times 512$, что улучшает анализ мелких объектов.
5. Слой Concat формирует $80 \times 80 \times 768$, соединяя $80 \times 80 \times 512$ с $80 \times 80 \times 256$.
6. C3k2 выдаёт $80 \times 80 \times 256$, углубляет признаки.
7. Слой Conv 256 преобразует $80 \times 80 \times 256$ в $40 \times 40 \times 256$ и подготавливает данные для следующего масштаба.
8. Concat соединяет $40 \times 40 \times 256$ с $40 \times 40 \times 512$ и формирует $40 \times 40 \times 768$.
9. Блок C3k2 выдаёт $40 \times 40 \times 512$, углубляет признаки.
10. Conv 512 преобразует $40 \times 40 \times 512$ в $20 \times 20 \times 512$.
11. Concat формирует $20 \times 20 \times 1536$, соединяя $20 \times 20 \times 512$ с $20 \times 20 \times 1024$.

12. C3k2 выдаёт 20x20x1024, углубляет признаки.

Neck готовит три карты признаков размерностями 80x80x256, 40x40x512 и 20x20x1024 и отправляет их в head для диагностики.

3.3.1.3 Head

Head – это последняя часть модели, которая выдаёт результаты на основе данных из neck. Блок Detect обрабатывает каждый масштаб и делает предсказания координат рамок, вероятностей классов и уверенности. Он использует свёртки, чтобы получить эти данные.

3.4 Структура модулей программы

Программа для диагностики заболеваний томатов представляет собой веб-приложение, реализованное с использованием Python, модели YOLO11 для обнаружения болезней, PostgreSQL для хранения данных, HTML/CSS и JavaScript для клиентского интерфейса. Структура программы организована в виде набора модулей, каждый из которых выполняет определённые функции. На схеме 3.4 представлена структура разрабатываемого решения.

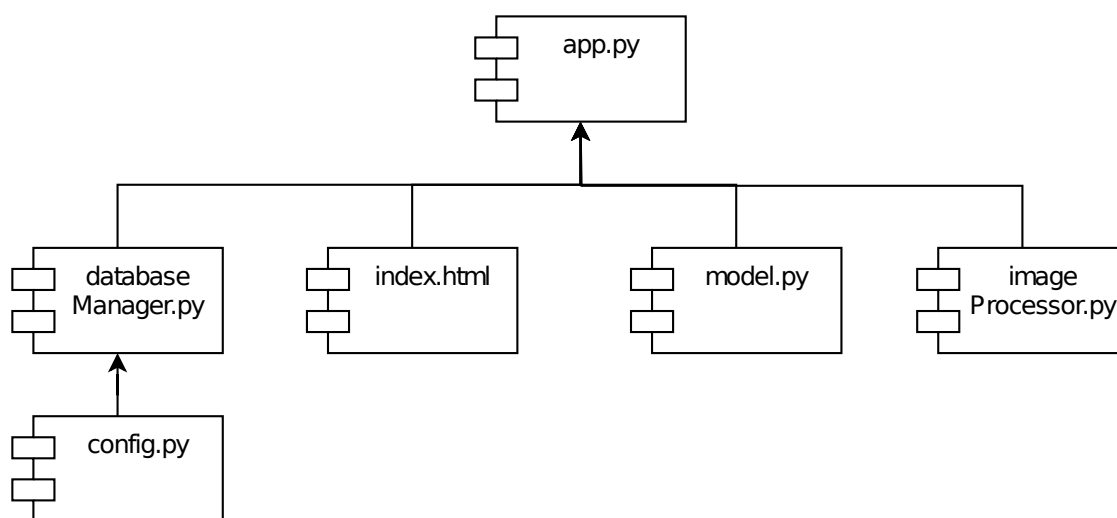


Рисунок 3.4 – Структура модулей программы

1. Модуль app.py – основной файл серверной части, реализованный на фреймворке Flask. Обрабатывает GET и POST запросы для загрузки изобра-

жений, кодирует изображения в base64 для отображения и запрашивает данные о болезни из базы данных.

2. Модуль `databaseManager.py` – файл для взаимодействия с базой данных PostgreSQL. Содержит класс `Database` для настройки соединения с базой, выполняет SQL-запросы для получения данных о болезни, включая методы лечения и профилактики.

3. Модуль `config.py` – файл конфигурации, содержащий параметры подключения к базе данных.

4. `index.html` – HTML-шаблон клиентского интерфейса. Содержит форму для загрузки изображений, блок для отображения данных о болезни. Использует CSS-стили из файла `styles.css`.

5. Модуль `model.py` загружает и настраивает обученную модель для анализ изображений и выявление заболеваний.

6. Модуль `imageProcessor.py` выполняет предварительную обработку изображений для анализа моделью и их преобразование для отображения в интерфейсе.

3.5 Структура базы данных

Сущности и отношения между ними отображены на ER-диаграмме.

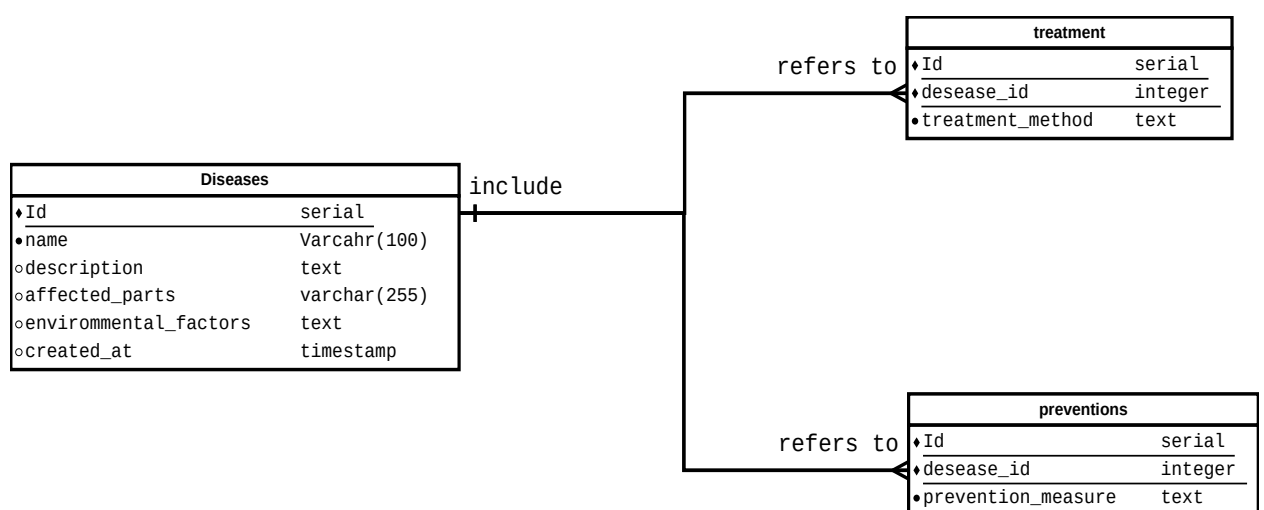


Рисунок 3.5 – ER-диаграмма

Таблица 3.1 – Атрибуты сущности diseases

Поле	Тип	Обязательное	Описание
1	2	3	4
id	serial	true	Первичный ключ
name	varchar(100)	true	Название болезни
description	text	false	Подробное описание болезни
affected_parts	varchar(255)	false	Части растения, поражаемые болезнью
environmental_factors	text	false	Экологические факторы, способствующие болезни
created_at	timestamp	false	Дата и время создания записи

Таблица 3.2 – Атрибуты сущности treatments

Поле	Тип	Обязательное	Описание
1	2	3	4
id	serial	true	Первичный ключ
disease_id	integer	true	Внешний ключ, ссылка на таблицу diseases
treatment_method	text	true	Описание методов лечения

Таблица 3.3 – Атрибуты сущности preventions

Поле	Тип	Обязательное	Описание
1	2	3	4
id	serial	true	Первичный ключ
disease_id	integer	true	Внешний ключ, ссылка на таблицу diseases
prevention_measure	text	true	Описание мер профилактики

4 Рабочий проект

4.1 Спецификация системы

4.1.1 Модуль databaseManager

Класс DatabaseManager реализует управление подключением к базе данных PostgreSQL и выполняет запросы для получения описания болезни, способы лечения и мер профилактики. В таблице 4.1 приведено описание методов класса.

Таблица 4.1 – Методы класса DatabaseManager

Название метода	Описание метода
configure_connection()	Настраивает параметры и устанавливает соединение с базой данных, возвращает объект соединения.
get_connection()	Создает и возвращает соединение с базой данных, используя параметры конфигурации из Config.
init_cursor(conn)	Инициализирует курсор с RealDictCursor для соединения и возвращает его.
fetch_disease_info(cursor, disease_name)	Выполняет запрос для получения описания болезни, возвращает словарь.
fetch_treatments(cursor, disease_id)	Получает методы лечения для болезни, возвращает список с ними.
fetch_preventions(cursor, disease_id)	Получает меры профилактики болезней, возвращает список с ними.
close_resources(cursor, conn)	Закрывает курсор и соединение.
get_disease(disease_name)	Используя предыдущие методы, извлекает из базы данных описание болезни, способы лечения и меры профилактики по названию.

4.1.2 Модуль imageProcessor

Класс ImageProcessor предназначен для подготовки изображений к анализу моделью и их отображению в веб-приложении. В таблице 4.2 приведено описание методов класса.

Таблица 4.2 – Методы класса ImageProcessor

Название метода	Описание метода
ensure_rgb_image(image)	Проверяет изображение и конвертирует в RGB, если необходимо.
resize_and_pad_image(image, target_size)	Изменяет размер изображения с сохранением пропорций и добавляет черные отступы.
enhance_image(image)	Применяет к изображению улучшения контрастности, резкости и цвета.
preprocess_image(image)	Используя предыдущие методы, изменяет размер изображения, добавляет отступы и улучшает качество для анализа моделью, возвращает обработанное изображение.
resize_for_display(image)	Изменяет размер изображения до фиксированного для отображения на веб-странице.
convert_to_bgr_array(image)	Конвертирует изображение в numpy массив и преобразует в BGR.
encode_to_base64(bgr_array)	Кодирует массив BGR в строку base64.
convert_to_base64(image)	Преобразует изображение в строку base64 для передачи и отображения в веб-приложении.

4.1.3 Модуль model

Класс TomatoDiseaseModel отвечает за загрузку модели YOLO и выполнение предсказаний болезней томатов на основе изображений. В таблице 4.3 приведено описание методов класса.

Таблица 4.3 – Методы класса TomatoDiseaseModel

Название метода	Описание метода
<code>predict(img_input)</code>	Анализирует входное изображение с помощью модели YOLO и возвращает результат.

4.1.4 Модуль trainModel

Класс YOLOTrainer предназначен для обучения и оценки модели YOLO. В таблице 4.4 приведено описание методов класса.

Таблица 4.4 – Методы класса YOLOTrainer

Название метода	Описание метода
<code>__init__(model_path, data_path, epochs, imgsz, batch)</code>	Инициализирует объект YOLOTrainer.
<code>load_model()</code>	Загружает модель YOLOv11.
<code>train()</code>	Выполняет обучение модели YOLOv11 с заданными гиперпараметрами.
<code>save_model(save_path)</code>	Сохраняет обученную модель по указанному пути.

Класс YOLOv11Inference отвечает за выполнение инференса модели YOLOv11 на тестовых изображениях. В таблице 4.5 приведено описание методов класса.

Таблица 4.5 – Методы класса YOLOv11Inference

Название метода	Описание метода
<code>__init__(model_path, device, class_labels, conf)</code>	Инициализирует обученную модель YOLOv11.
<code>run(image_dir, n_ims, rows)</code>	Выполняет предсказание на случайных изображениях из директории.

Продолжение таблицы 4.5

Название метода	Описание метода
<code>predict_single(image_path)</code>	Выполняет предсказание на одном изображении.
<code>process_results(results)</code>	Извлекает из результата рамки, метки и уверенности.
<code>visualize_results(res, n_imgs, rows)</code>	Визуализирует результаты предсказания.

4.1.5 Модуль app

Модуль app управляет веб-приложением Flask, обрабатывает загрузку изображений, вызывает предсказания модели и отображает результаты в веб-интерфейсе. В таблице 4.6 приведено описание методов модуля.

Таблица 4.6 – Методы модуля app

Название метода	Описание метода
<code>save_image_to_tempfile(image)</code>	Сохраняет изображение во временный файл.
<code>predict_with_fallback(image, temp_path)</code>	Выполняет повторное предсказание для иначе обработанного изображения.
<code>prepare_image_for_display(image)</code>	Подготавливает изображение для отображения.
<code>process_image(image)</code>	Обрабатывает изображение для предсказания и отображения.
<code>handle_get_request()</code>	Обрабатывает GET-запрос.
<code>check_file_presence()</code>	Проверяет наличие файла в POST-запросе.
<code>process_uploaded_file(file)</code>	Обрабатывает загруженный файл, возвращает информацию о болезни.
<code>handle_post_request()</code>	Обрабатывает POST-запрос с загрузкой файла.

Продолжение таблицы 4.6

Название метода	Описание метода
upload_file()	Обработывает GET и POST запросы, рендерит шаблон index.html с данными.

4.2 Обучения модели

4.2.1 Описание датасета

Обучение и тестирование модели проводились на специально подготовленном датасете, содержащем изображения здоровых растений и листьев томатов с признаками различных заболеваний. Общее количество изображений равно 18612. Из них 70% используется для обучения, 20% для валидации и 10% для тестирования. Изображения приводятся к размеру 640x640 пикселей. Большая часть данных была взята из открытого датасета plantvillage, остальные фото были найдены в сети интернет.

Датасет был подготовлен с использованием разметки в формате YOLO, где каждое изображение сопровождается аннотацией с координатами ограничивающих рамок и метками классов.

Для улучшения качества модели применялись методы аугментации данных, такие как повороты и изменения яркости, что позволило увеличить разнообразие обучающей выборки.

4.2.2 Результаты обучения

Обучение модели проводилось на платформе Google Colab с использованием графического процессора. Обучение проводилось на протяжении 100 эпох с размером пакета равным 16. Начальная скорость обучения равна 0.001. Для оптимизации использовался оптимизатор AdamW. Коэффициент импульса равен 0.937. Снижение веса для регуляции установлено в 0.0005.

Графики на рисунке 4.1 отражают динамику метрик в процессе обучения. На оси x отложены эпохи (от 1 до 100), а на оси y – значения метрик.

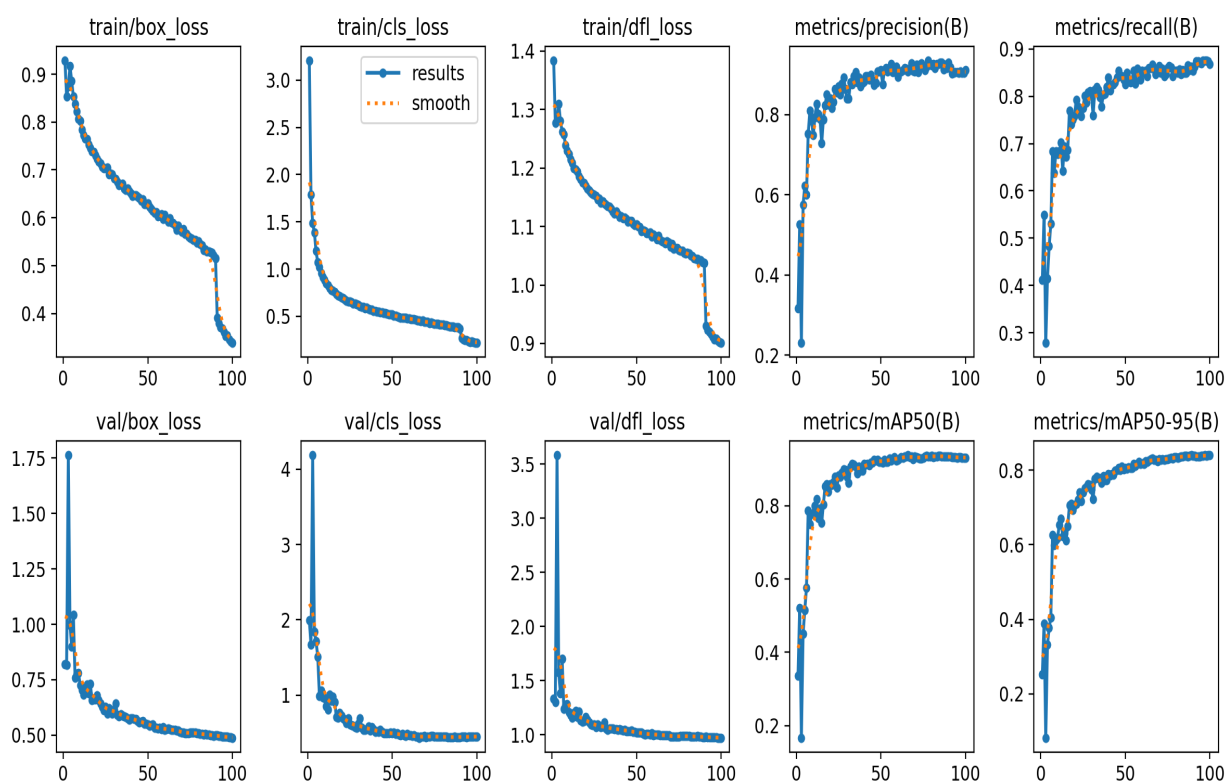


Рисунок 4.1 – Динамика метрик обучения.

Box Loss (train/val) показывает потери на ограничивающие рамки, то есть насколько точно модель обводит большие участки листьев. На старте потери были высокими: больше 0.9 для обучающего набора и 1.75 для валидационного. Но к 50-й эпохе они снизились до 0.6 (train) и 0.6 (val), а к концу обучения, на 100-й эпохе, достигли 0.3 и 0.5 соответственно. Это говорит о том, что модель стала гораздо точнее определять местоположение поражённых зон.

Class Loss (train/val) отражает потери классификации. В начале они превышали 3 при обучении и 4 при валидации, но уже к 30-й эпохе упали до 0.7 и 0.6 соответственно. К 100-й эпохе значения снизились, примерно, до 0.2 при обучении и 0.3 при валидации. Можно сделать вывод, что модель научилась уверенно определять болезни на фото.

Dfl Loss (train/val) показывают, как изменялись потери, связанные с распределённой фокальной функцией потерь (dfl Loss), которая отвечает за точное определение формы и размеров ограничивающих рамок вокруг больных участков листьев. В начале они составляли 1.4 при обучении и 3.5 при

валидации. К 30-й эпохе значение снизилось до 1.1 и 1.2. К концу обучения значения снизились до, примерно, 0.9 при обучении и 1 при валидации. Это говорит о том, что модель научилась более точно подгонять рамки под форму и размеры больных участков.

Precision(точность) иллюстрирует, насколько часто модель правильно классифицирует больные участки листьев, избегая ложных срабатываний. На старте точность была низкой – около 0.2, что говорит о значительном количестве ошибок. Однако уже к 20-й эпохе значение выросло до 0.6, а к 50-й – до 0.8. К 80-й эпохе точность стабилизировалась на уровне 0.85, оставаясь на этом уровне до конца обучения. Это указывает на то, что модель научилась с высокой уверенностью определять болезни.

Recall(полнота) показывает, насколько успешно модель обнаруживает все случаи заболеваний. Начальное значение было около 0.3. К 50-й эпохе полнота поднялась до 0.8. К концу обучения она достигла 0.88. Это говорит о том, что модель стала эффективно находить почти 88% всех больных участков.

mAP50 отражает среднюю точность классификации при пороге пересечения IoU равного 0.5. На первой эпохе mAP50 был меньше 0.2. К 20-й эпохе значение выросло до 0.6, а к 50-й – до 0.83. К 80-й эпохе mAP50 стабилизировался на уровне 0.88. Данный результат подтверждает высокую способность модели правильно классифицировать заболевания.

mAP50-95 учитывает среднюю точность при порогах IoU от 0.5 до 0.95, оценивая как классификацию, так и точность локализации. Начальное значение было около 0.15, но к 20-й эпохе оно стало больше 0.6, а к 50-й – превысило 0.7. К 100-й эпохе mAP50-95 поднялся до 0.8. Это показывает, что модель не только верно определяет болезни, но и достаточно точно обводит поражённые области, даже при высоких требованиях к точности.

В целом, данные результаты свидетельствуют об успешном процессе обучения модели, демонстрируя постепенное повышение ключевых показателей эффективности на протяжении всех итераций.

4.3 Тестирование модели

После завершения обучения модели на подготовленном наборе данных, был проведён этап её тестирования. Для анализа были использованы изображения, которые модель ранее не видела. Это позволит оценить её способность обобщать знания. В процессе тестирования были собраны данные о точности классификации. Результаты представлены в виде наглядных графиков и матриц, которые отражают сильные и слабые стороны модели.

4.3.1 Анализ F1-меры в зависимости от порога уверенности

График 4.2 демонстрирует зависимость F1-меры от порога уверенности модели. Анализ представления позволяет определить оптимальный порог, при котором модель демонстрирует наилучшее качество классификации как для отдельных классов, так и для всех категорий в целом.

На оси x отложены значения порога уверенности, а на оси y – F1-меры. Цветные кривые соответствуют одному из классов заболеваний томатов.

На графике видно, что максимальное оптимальное значение F1 достигается при пороге уверенности 0.444 и равно оно 0.89.

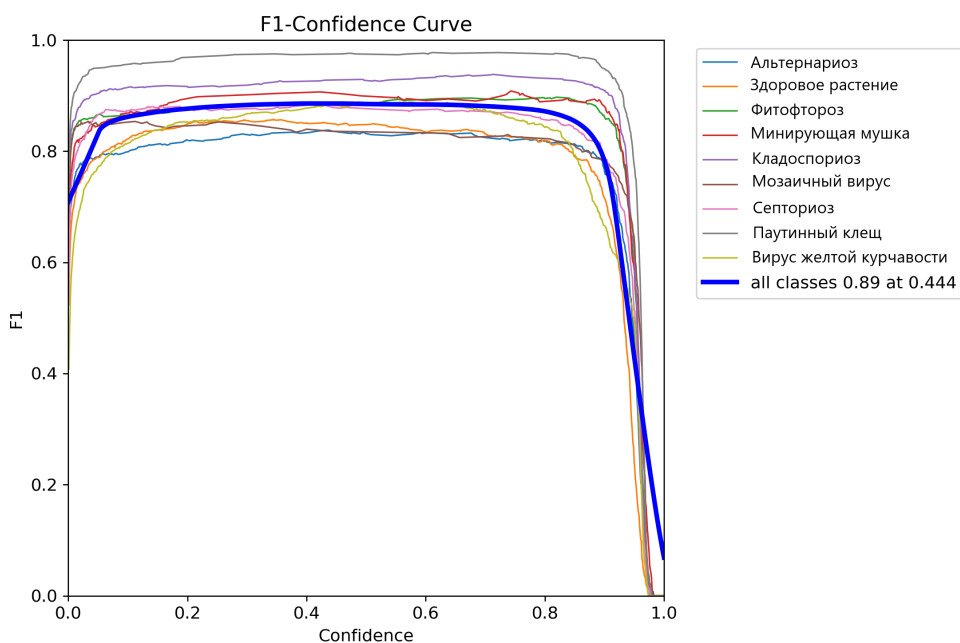


Рисунок 4.2 – Кривые F1-меры в зависимости от порога уверенности.

4.3.2 PR-кривые: точность и полнота по классам

На изображении 4.3 представлен график кривых точности и полноты, который используется для оценки качества модели классификации.

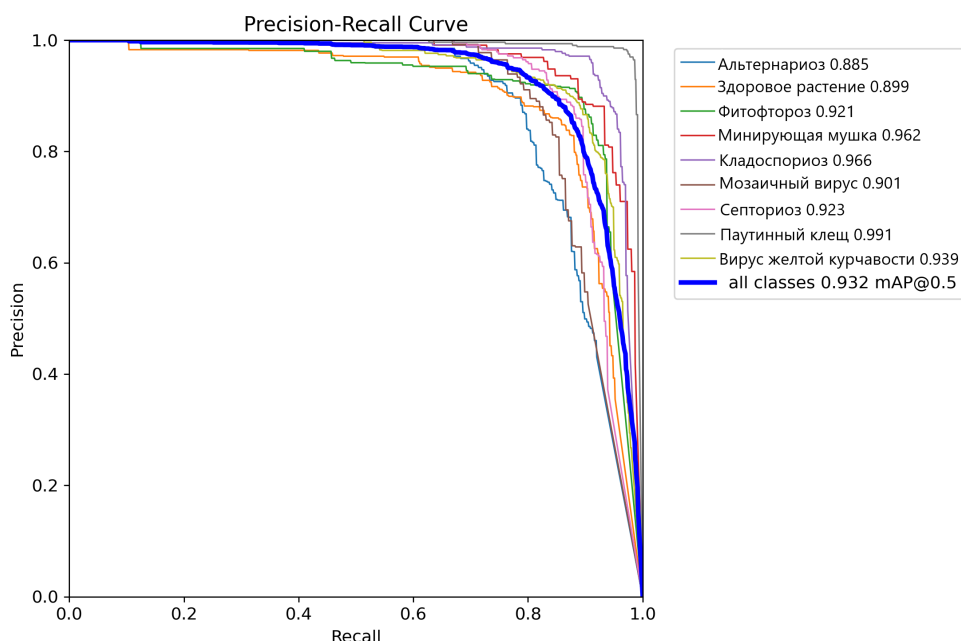


Рисунок 4.3 – Кривые точности и полноты

По оси X отложена полнота (Recall) – доля правильно найденных положительных примеров от всех реально положительных. По оси Y отложена точность (Precision) – доля правильно классифицированных положительных примеров от всех предсказанных как положительные.

Каждая кривая представляет собой метрику качества предсказаний для одного из классов. Чем выше её расположена и ближе к правому верхнему углу, тем лучше модель справляется с классификацией соответствующего класса.

Ниже приведено описание для каждого класса:

- альтернариоз: демонстрирует хорошее качество с точностью 0.885, но немного уступает другим классам по устойчивости на высоких значениях recall;
- здоровое растение: имеет точность 0.899, что указывает на уверенность модели при классификации здоровых листьев;

- фитофтороз: с точностью 0.921, модель уверенно различает этот класс, кривая остаётся высокой даже при увеличении полноты;
- минирующая мушка: точность 0.962, одна из наиболее уверенных кривых – модель чётко распознаёт повреждения, вызванные этим вредителем;
- кладоспориоз: высокая точность 0.966, кривая близка к верхней границе, что говорит о хороших результатах классификации для этого класса;
- мозаичный вирус: точность 0.901, хорошая предсказательная способность модели, хотя немного ниже, чем у большинства других заболеваний;
- септориоз (розовая кривая): точность 0.923, стабильная кривая указывает на уверенное распознавание симптомов данного заболевания;
- паутинный клещ: точность 0.991 – наивысший показатель среди всех классов, кривая практически идеально приближена к верхней границе;
- вирус жёлтой курчавости: точность 0.939, модель уверенно справляется с его распознаванием, несмотря на сложность визуальных признаков.

Жирная синяя линия показывает усреднённое поведение модели по всем классам. Среднее значение точности при полноте 0.5 составляет 0.932, что свидетельствует о высоком качестве классификации.

В целом, график демонстрирует стабильную и высокую производительность классификатора по всем классам заболеваний.

4.3.3 Нормализованная матрица ошибок

На рисунке 4.4 представлена нормализованная матрица ошибок (confusion matrix). Она позволяет оценить, насколько точно модель выполняет классификацию, а также выявить типичные ошибки классификации. По горизонтальной оси отложены истинные классы (True), а по вертикальной – предсказанные классы (Predicted). Цветовая шкала справа варьируется от 0.0 (светло-голубой) до 0.8 (тёмно-синий). Она отражает долю правильных или ошибочных предсказаний.

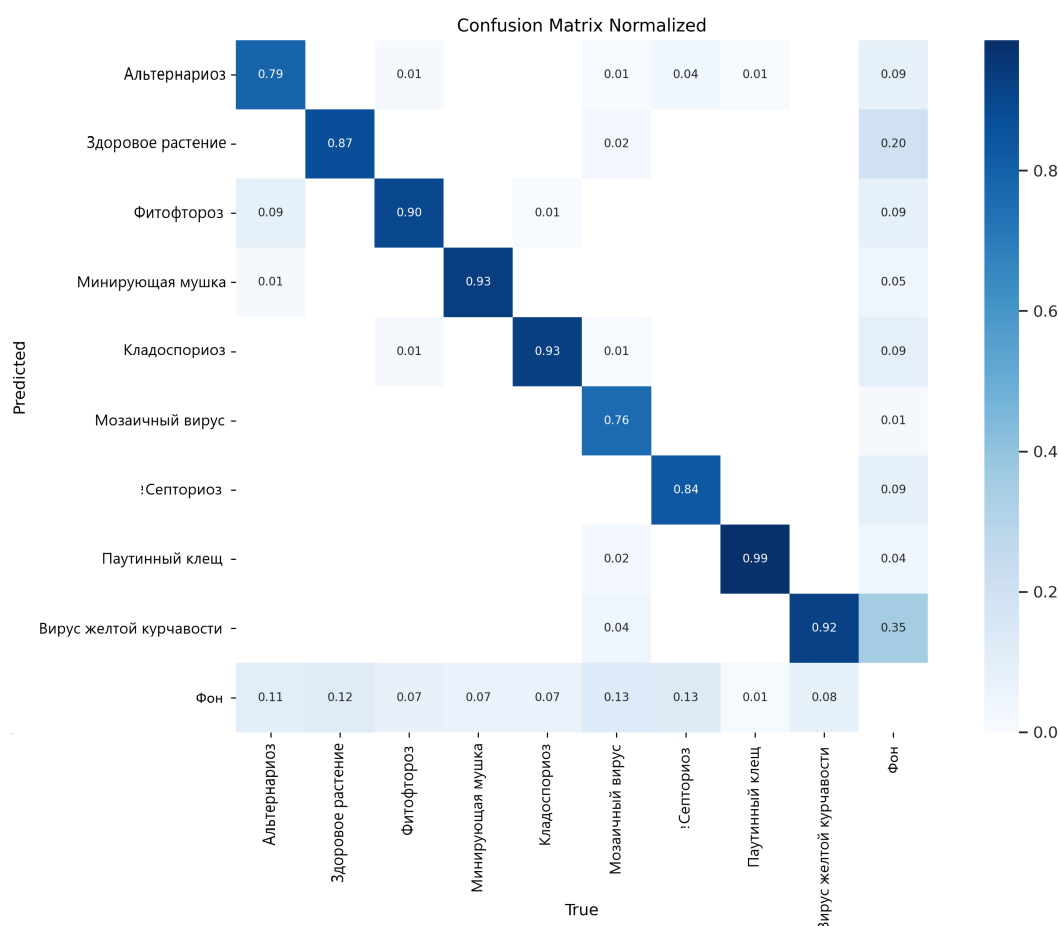


Рисунок 4.4 – Нормализованная матрица ошибок

Модель корректно классифицировала 79% изображений, относящихся к «Альтернариозу». Однако в 9% случаев изображения с этим заболеванием были ошибочно отнесены к категории «Фон», что может быть связано с недостаточной выраженностью визуальных признаков или с фоном, схожим по текстуре. Небольшое число ошибок (по 1-4%) приходится на классы «Здоровое растение», «Фитофтороз», «Кладоспориоз» и «Септориоз».

Класс «Здоровое растение» был определён правильно в 87% случаев. Основной источник ошибок – это отнесение к классу «Фон» (20%), что может говорить о том, что визуально здоровые листья иногда не отличимы от фона. Также небольшая доля изображений ошибочно классифицирована как «Фитофтороз» (2%).

Для класса «Фитофтороз» модель продемонстрировала высокую точность – 90% правильных классификаций. Тем не менее, 9% изображений с фитофторозом были отнесены к «Альтернариозу». Это может быть вызвано

сходством визуальных симптомов этих болезней. Остальные ошибки незначительны.

В 93% случаев изображения с «Минирующей мушкой» были определены верно. Наиболее частая ошибка – путаница с «Альтернариозом» в 1% случаев и с «Фоном» в 5%, что может объясняться слабой выраженностью повреждений или их перекрытием с фоновыми элементами.

Модель правильно распознала 93% изображений с «Кладоспориозом». Основная ошибка – 1% спутан с «Фитофторозом» и 1% – с «Септориозом». Кроме того, 9% изображений были отнесены к «Фону», что также может указывать на недостаточную выраженность симптомов.

Результаты по «Мозаичному вирусу» оказались менее точными – 76% правильных классификаций. Существенная доля изображений (13%) была отнесена к «Фону», и ещё часть спутана с «Здоровыми растениями» и другими классами. Причиной этого является визуальная неоднозначность визуальных признаков вируса.

Модель правильно определила 84% изображений с «Септориозом». Ошибки классификации связаны с отнесением 9% изображений к «Фону» и небольшими путаницами с «Фитофторозом» и другими заболеваниями из-за визуального сходства симптомов.

Класс «Паутинный клещ» распознавался почти безошибочно: 99% изображений были классифицированы правильно. Небольшие ошибки (до 2%) возникали при отнесении к «Септориозу» и «Фону», что может быть связано с лёгкой спутанностью паутины с текстурой листа.

Вирус жёлтой курчавости: модель показала высокую точность – 92% правильных классификаций. Однако значительное количество изображений (35%) было ошибочно отнесено к «Фону», что говорит о сложности выделения признаков вируса на изображениях с низкой контрастностью или слабой выраженностью симптомов.

4.4 Тестирование интеллектуальной системы

На рисунке 4.5 показана основная страница интеллектуальной системы.

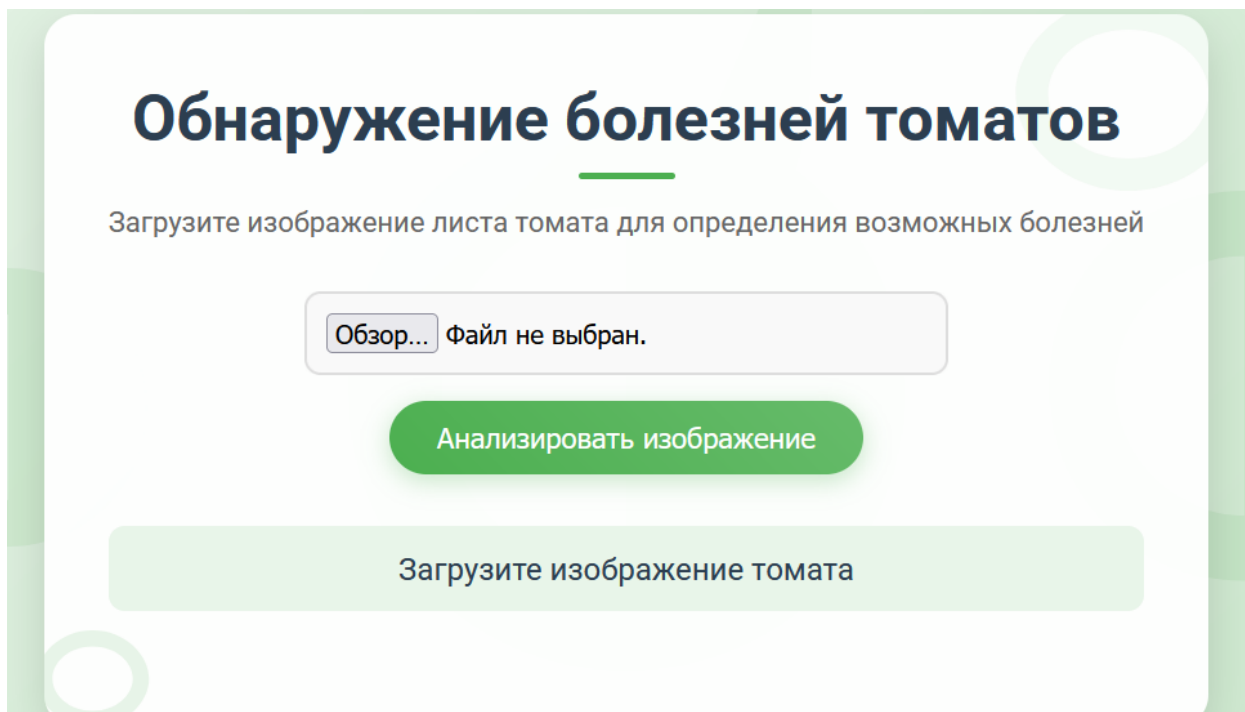


Рисунок 4.5 – Основная страница системы

На рисунке 4.6 показан выбор фотографии, а на 4.7 результат выбора.

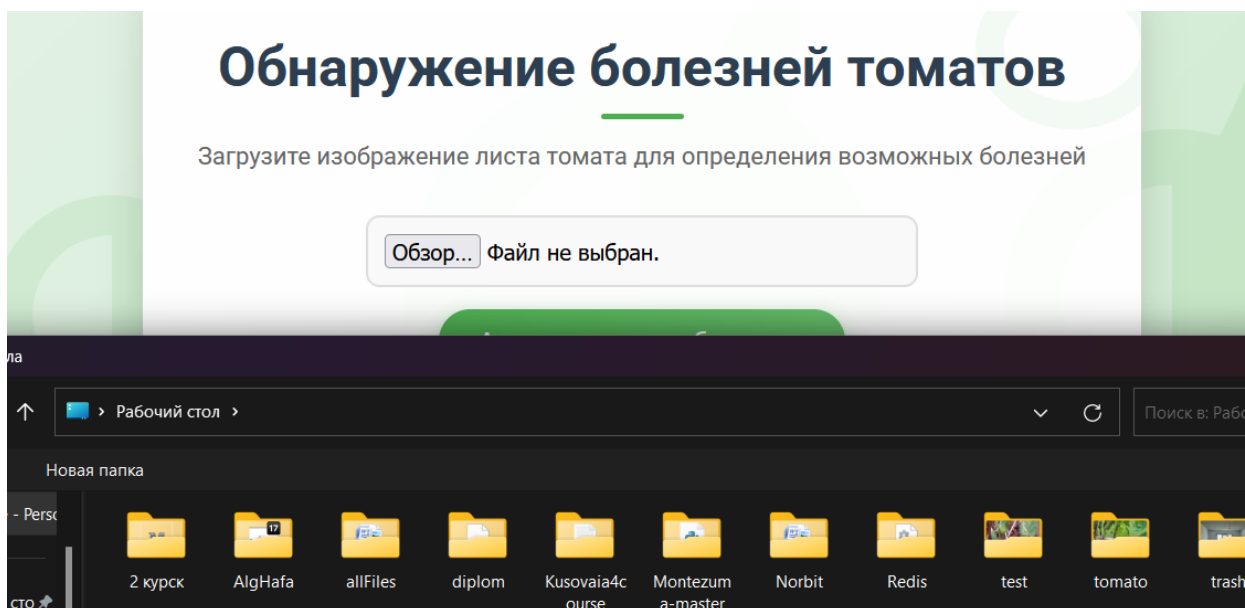


Рисунок 4.6 – Выбор изображения

Обнаружение болезней томатов

Загрузите изображение листа томата для определения возможных болезней

Обзор... альтерниоз tomatov.jpg

Анализировать изображение

Загрузите изображение томата

Рисунок 4.7 – Результат выбора

На рисунке 4.8 показан результат анализа фотографии томата с признаками альтернариоза.

Обнаруженная болезнь: Альтерниоз (92.98% уверенности)



Рисунок 4.8 – Результат анализа фотографии томата с признаками альтернариоза

На рисунке 4.9 показан вывод описания альтернариоза.

Альтернариоз

Описание: Альтернариоз, вызываемый грибом *Alternaria solani*, является распространенным заболеванием томатов. На нижних листьях появляются темные пятна с концентрическими кольцами, напоминающими мишень. Эти пятна увеличиваются, вызывая пожелтение и отмирание листьев, что снижает фотосинтез и урожайность. В запущенных случаях болезнь может поражать стебли и плоды, образуя темные вдавленные участки. Альтернариоз прогрессирует в теплую (25-30°C) и влажную погоду, особенно после дождей или чрезмерного полива. Болезнь распространяется через споры, переносимые ветром или брызгами воды.

Рисунок 4.9 – Описание альтернариоза

На рисунке 4.10 описано лечение заболевания.

Лечение:

- ✓ Немедленное удаление и уничтожение пораженных частей растения для предотвращения распространения спор.
- ✓ Применение фунгицидов, таких как бордоская жидкость или препараты на основе меди, каждые 7-10 дней при первых признаках болезни.
- ✓ Для органического выращивания использование биопрепаратов, таких как Фитоспорин-М или серосодержащие средства.
- ✓ Улучшение циркуляции воздуха путем прореживания кустов и удаления нижних листьев.
- ✓ В тяжелых случаях консультация с агрономом для применения системных фунгицидов, таких как Ордан.

Рисунок 4.10 – Лечение альтернариоза

На рисунке 4.11 описано меры профилактики.

Профилактика:

- ✓ Использование устойчивых сортов томатов, таких как Фитоус или Титан.
- ✓ Соблюдение севооборота, избегая посадки томатов или родственных культур на одном и том же месте в течение 2-3 лет.
- ✓ Использование мульчи для предотвращения брызг почвы на листья.
- ✓ Полив у основания растений, чтобы избежать намокания листьев.
- ✓ Обеспечение хорошей вентиляции путем правильного расстояния между растениями и подвязки.
- ✓ Удаление и уничтожение всех растительных остатков в конце сезона.

Рисунок 4.11 – Меры профилактики альтернариоза

На рисунке 4.12 показан случай, когда загружается фото другого растения, отличного от томата.

Не удалось обнаружить куст томата на изображении



Рисунок 4.12 – Анализ фотографии ракиты

На рисунке 4.13 показан случай, когда пользователь нажимает на кнопку "Анализировать изображение не загрузив предварительно фото.

Обнаружение болезней томатов

Загрузите изображение листа томата для определения возможных болезней

Обзор... Файл не выбран.

Пожалуйста, выберите файл.

изображение

Загрузите изображение томата

Рисунок 4.13 – Уведомление, что нужно выбрать файл перед анализом

На рисунке 4.14 показано успешное определение фитофтороза .

Обнаруженная болезнь: Фитофтороз (96.14% уверенности)



Рисунок 4.14 – Результат загрузки фото с фитофторозом

На рисунке 4.15 показано успешное определение минирующей мушки.

Обнаруженная болезнь: Листовая минирующая мушка (96.24% уверенности)



Рисунок 4.15 – Результат загрузки фото с минирующей мушкой

На рисунке 4.16 показано успешное определение кладоспориоза.

Обнаруженная болезнь: Кладоспориоз (90.22% уверенности)



Рисунок 4.16 – Результат загрузки фото с кладоспориозом

На рисунке 4.17 показано успешное определение мозаичного вируса.

Обнаруженная болезнь: Мозаичный вирус (95.14% уверенности)



Рисунок 4.17 – Результат загрузки фото с мозаичным вирусом

На рисунке 4.18 показано успешное определение септориоза.

Обнаруженная болезнь: Септориоз (93.89% уверенности)



Рисунок 4.18 – Результат загрузки фото с септориозом

На рисунке 4.19 показано успешное определение паутинного клеща.

Обнаруженная болезнь: Паутинный клещ (96.33% уверенности)



Рисунок 4.19 – Результат загрузки фото с паутинным клещом

На рисунке 4.20 показано успешное определение вируса желтой курчавости листьев.

Обнаруженная болезнь: Вирус желтой курчавости листьев (91.83% уверенности)



Рисунок 4.20 – Результат загрузки фото с вирусом желтой курчавости

На рисунке 4.21 показано успешное определение здорового томата.

Здоровое растение (94.68% уверенности)



Рисунок 4.21 – Результат загрузки фото здорового томата

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной работы была разработана интеллектуальная система для выявления заболеваний сельскохозяйственных растений, в частности томатов, с использованием технологий машинного обучения и веб-разработки.

Основные результаты работы:

1. Проведен анализ предметной области. Рассмотрены основные заболевания томатов, современные технологий, которые могут использоваться для их диагностики. Проанализированы ограничения традиционных методов обнаружения заболеваний и преимущества применения ИИ.
2. Спроектирована структура интеллектуальной системы. Определены функциональные и нефункциональные требования.
3. Реализована и обучена свёрточная нейронная сеть на базе модели Yolo11. CNN демонстрирует хороший результат, достигая точности mAP50-95 более 0.8.
4. Разработано веб-приложение. Оно позволяет загружать изображения, получать результаты анализа и рекомендации по лечению.
5. Проведено тестирование системы. Оно подтвердило высокую точность модели с F1-мерой 0.89 и средней точностью 0.932 на тестовом наборе данных. Проверена функциональность веб-приложения.

Все требования, объявленные в техническом задании, были полностью реализованы, все задачи, поставленные в начале разработки проекта, были также решены.

Разработанная система представляет собой эффективный инструмент, который может помочь оптимизировать процесс мониторинга здоровья растений и минимизировать потери урожая.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ходж, Д. Ботаника для садоводов / Д. Ходж – Москва : Азбука-Бизнес, 2024. - 224 с. – ISBN 978-5-389-13415-7. – Текст : непосредственный.
2. Кошкин, Е. И. Патофизиология сельскохозяйственных культур / Е. И. Кошкин – Москва : РГ-Пресс, 2024. - 304 с. – ISBN 978-5-9988-1405-1. – Текст : непосредственный.
3. Сергеева, М. Н. Культурные растения / М. Н. Сергеева – Москва : Проспект, 2023. - 120 с. – ISBN 978-5-392-37362-8. – Текст : непосредственный.
4. Абросимов, В.К. Интеллектуальные сельскохозяйственные роботы- /В.К. Абросимов, А.Н. Райков – Москва : Карьера Пресс, 2022. - 512 с. – ISBN 978-5-00074-318-8. – Текст : непосредственный.
5. Гриценко, В. В. Меры борьбы с болезнями и вредителями растений в сооружениях защищенного грунта /В. В. Гриценко, И. М. Митюшев, Ю. М. Стройков – Москва : Академия, 2020. - 160 с. – ISBN 978-5-44-689434-5. – Текст : непосредственный.
6. Ерофеева, Т. В. Сельскохозяйственная экология /Ерофеева Т. В., Фадькин Г. Н., Чурилова В. В. Санкт-Петербург : Лань, 2025. - 100 с. – ISBN 978-5-507-52249-1. – Текст : непосредственный.
7. Курбанов, С. А. Сельскохозяйственная мелиорация /С. А. Курбанов – Санкт-Петербург : Лань, 2021. - 208 с. – ISBN 978-5-8114-6623-8. – Текст : непосредственный.
8. Овсинский, И. Е. Новая система земледелия /И. Е. Овсинский – Москва : Концептуал, 2023. - 240 с. – ISBN 978-5-907624-99-3. – Текст : непосредственный.
9. Современное сельское хозяйство : сайт / fnb TECH. – Москва : fnb TECH, 2025 – . – URL: <https://fnb.tech/ru/modern-agriculture/> (дата обращения: 23.05.2025). – Текст: электронный.
10. Монторо, Ж. Семена и зерна / Ж. Монторо – Москва : Эксмо, 2024. - 256 с. – ISBN 978-5-04-196329-3. – Текст : непосредственный.

11. Копытин, И. П. Ведение сельского хозяйства в Центрально-Нечерноземном округе России / И. П. Копытин – Санкт-Петербург : Лань, 2022. - 336 с. – ISBN 978-5-8114-9863-5. – Текст : непосредственный.
12. Савельев, В. А. Растениеводство / В. А. Савельев – Санкт-Петербург : Лань, 2021. - 316 с. – ISBN 978-5-8114-8194-1. – Текст : непосредственный.
13. Ториков, В. Е. Агрономический контроль в растениеводстве / В. Е. Ториков, О. В. Мельникова/ Санкт-Петербург :Лань, 2024. - 132 с. - ISBN 978-5-507-49427-9. - Текст: непосредственный.
14. Ганичкина, О. А. Секреты огородника. Как получить богатый урожай овощей и зелени на вашем участке / О. А. Ганичкина, А. В. Ганичкин / Москва :Эксмо, 2024. - 224 с. - ISBN 978-5-04-211846-3. - Текст: непосредственный.
15. Кочелаева, Л. Н. Сеньор Помидор. Выращиваем, ухаживаем и едим / Л. Н. Кочелаева – Москва : АСТ, 2024. - 160 с. – ISBN 978-5-17-161963-3. – Текст : непосредственный.
16. Кузнецова, Е. А. Рассадоводство. Первые шаги к здоровому урожаю / Е. А. Кузнецова – Москва: Издательство АСТ, 2023. - 160 с. – ISBN 978-5-17-157334-8. – Текст: непосредственный.
17. Ожехелева, З. Е. Биологические препараты для эффективного садоводства / З. Е. Ожехелева – Москва: ФГБНУ ВНИИСПК, 2022. - 166 с. – ISBN 978-5-6049204-3-5. – Текст: непосредственный.
18. Кизима, Г. А. Огород и сад для ленивых. Урожаю быть / Г. А. Кизима – Москва: Издательство АСТ, 2020. - 192 с. – ISBN 978-5-17-120651-2. – Текст: непосредственный.
19. Волкова, А. П. Энциклопедия пасленовых. Томат. Перец. Баклажан. Физалис / А. П. Волкова – Москва: Издательство АСТ, 2024. - 416 с. – ISBN 978-5-17-164151-1. – Текст: непосредственный.
20. Кизима, Г. А. Рассада для начинающих. Первые шаги к богатому урожаю / Г. А. Кизима – Москва: Издательство АСТ, 2025. - 128 с. – ISBN 978-5-17-164591-5. – Текст: непосредственный.

21. Демиденко, А. Введение в Computer Vision: как научить компьютер видеть / А. Демиденко – Москва: Литрес, 2025. – 100 с. – ISBN 978-5-04-720541-0. – Текст: непосредственный.
22. Рутковская, Д. Нейронные сети, генетические алгоритмы и нечеткие системы / Д. Рутковская, М. Пилиньский, Л. Рутковский – Москва: Горячая линия-Телеком, 2023. – 385 с. – ISBN 978-5-9912-0320-3. – Текст: непосредственный.
23. Лекун, Я. Как учится машина: Революция в области нейронных сетей и глубокого обучения / Я. Лекун – Москва: Альпина PRO, 2021. – 335 с. – ISBN 978-5-907394-92-6. – Текст: непосредственный.
24. Мэтиз, Э. Изучаем Python: программирование игр, визуализация данных, веб-приложения. 3-е издание / Э. Мэтиз – Санкт-Петербург: БХВ, 2024. – 352 с. – ISBN 978-5-9775-1944-1. – Текст: непосредственный.
25. Демиденко, А. ИИ и зрение: как машины понимают изображения / А. Демиденко – Москва: Литрес, 2025. – 80 с. – ISBN 978-5-04-727123-1. – Текст: непосредственный.
26. Любанович, Б. Простой Python / Б. Любанович – Санкт-Петербург: Питер, 2021. – 592 с. – ISBN 978-5-4461-1701-7. – Текст: непосредственный.
27. Амундсен, М. RESTful Web API паттерны и практики / А. Демиденко – Москва: Sprint Book, 2025. – 464 с. – ISBN 978-601-08-4867-2. – Текст: непосредственный.
28. Определитель болезней томатов: фото, описание, меры борьбы и профилактика : сайт / Огород. – Москва : Огород, 2023 – . – URL: <https://www.ogorod.ru/ru/ogorod/tomats/14384/Opredelitel-bolezney-tomato-foto-opisaniye-mery-borby-i-profilaktika.htm> (дата обращения: 13.05.2025). – Текст: электронный.
29. Болезни рассады томатов: описание с фото, лечение : сайт / КП. – Москва : КП, 2023 – . – URL: <https://www.kp.ru/family/sad-i-ogorod/bolezni-rassady-tomato/> (дата обращения: 13.05.2025). – Текст: электронный.
30. Распознавание объектов с помощью YOLO v3 на Tensorflow 2.0 : сайт / Proglib. – Москва : Proglib, 2020 – . – URL: <https://proglib.io/p/>

raspoznavanie-obektov-s-pomoshchyu-yolo-v3-na-tensorflow-2-0-2020-11-08 (дата обращения: 13.04.2025). – Текст: электронный.

31. YOLOv11: улучшения в детекции объектов для реального времени : сайт / Ultralytics, 2024 – . – URL: <https://docs.ultralytics.com/ru/models/yolo11/> (дата обращения: 13.05.2025). – Текст: электронный.

32. Детекция объектов с помощью YOLOv5 : сайт / Habr. – Москва : Habr, 2021 – . – URL: <https://habr.com/ru/articles/576738/> (дата обращения: 11.05.2025). – Текст: электронный.

33. YOLOv7: пользовательское обнаружение объектов : сайт / Habr. – Москва : Habr, 2022 – . – URL: <https://habr.com/ru/articles/700794/> (дата обращения: 11.05.2025). – Текст: электронный.

34. Обнаружение объектов с YOLOv3 на Tensorflow 2.0 : сайт / Habr. – Москва : Habr, 2021 – . – URL: <https://habr.com/ru/articles/556404/> (дата обращения: 21.04.2025). – Текст: электронный.

35. Как выполнить обнаружение объектов YOLO с помощью OpenCV и PyTorch в Python : сайт / Waksoft. – Челябинск : Waksoft, 2021 – . – URL: <https://waksoft.susu.ru/2021/05/19/kak-vypolnit-obnaruzhenie-obektov-yolo-s-pomoshhyu-opencv-i-pytorch-v-python/> (дата обращения: 11.05.2025). – Текст: электронный.

36. Explore Ultralytics YOLOv11 : сайт / YOLOv11, 2024 – . – URL: <https://yolov11.com/> (дата обращения: 21.04.2025). – Текст: электронный.

37. Как использовать YOLOv11 для обнаружения объектов : сайт / SO Development. – Москва : SO Development, 2024 – . – URL: <https://ru.so-development.org/how-to-use-yolov11-for-object-detection/> (дата обращения: 11.05.2025). – Текст: электронный.

38. YOLO11: A New Iteration of “You Only Look Once”: сайт / viso.ai, 2024 – . – URL: <https://viso.ai/computer-vision/yolov11/> (дата обращения: 12.05.2025). – Текст: электронный.

39. YOLOv11: настройка и обучение модели для специфичных задач : сайт / Roboflow, 2024 – . – URL: <https://roboflow.com/model/yolo11> (дата обращения: 12.05.2025). – Текст: электронный.

40. Документация Python: сайт / Habr. – Москва : Habr, 2024 – . – URL: <https://www.python.org/doc/> (дата обращения: 02.03.2025). – Текст: электронный.
41. Язык программирования Python: особенности и перспективы : сайт / GeekBrain. – Москва : GeekBrain, 2024 – . – URL: <https://gb.ru/blog/python/> (дата обращения: 14.05.2025). – Текст: электронный.
42. Обзор языка программирования Python : сайт / Proglib. – Москва : Code Basics, 2023 – . – URL: https://code-basics.com/ru/blog_posts/obzor-yazyka-programmirovaniya-python (дата обращения: 11.03.2025). – Текст: электронный.
43. Библиотеки для веб-разработки на Python : сайт / Code Basics. Skyprow – Москва : Skyprow, 2023 – . – URL: <https://sky.pro/wiki/python/biblioteki-dlya-veb-razrabotki-na-python/> (дата обращения: 11.03.2025). – Текст: электронный.
44. Google Colaboratory : сайт / Google Colab, 2025 – . – URL: <https://colab.google/> (дата обращения: 24.03.2025). – Текст: электронный.
45. Мега-Учебник Flask : сайт / Habr. – Москва : Habr, 2024 – . – URL: <https://habr.com/ru/articles/804245/> (дата обращения: 12.03.2025). – Текст: электронный.
46. Проектирование RESTful API с помощью Python и Flask : сайт / Skyprow. – Москва : Skyprow, 2024 – . – URL: <https://sky.pro/media/kak-sozdat-rest-api-na-flask/> (дата обращения: 12.03.2025). – Текст: электронный.
47. Документация Flask : сайт / Flask, 2020 – . – URL: <https://flask.palletsprojects.com/en/stable/> (дата обращения: 14.05.2025). – Текст: электронный.
48. Vatathanavaro, S. White Blood Cell Classification: A Comparison between VGG-16 and ResNet-50 Models / V. ESupawit, T. Suchat, P. Kitsuchart – Bangkok : PG- King Mongkut's Institute of Technology Ladkrabang, 2018. - 2 с. – Текст : электронный – . – URL: <https://site.ieee.org/thailand-cis/files/2018/11/JSCI6-Paper-2.pdf> (дата обращения: 11.05.2025).

49. Фреймворк Flask: как он работает и зачем нужен : сайт / Skillbox. – Москва : Skillbox, 2023 – . – URL: <https://skillbox.ru/media/code/freymvork-flask-kak-on-rabotaet-i-zachem-nuzhen/> (дата обращения: 14.05.2025). – Текст: электронный.

50. Документация PostgreSQL : сайт / PostgreSQL, 2025 – . – URL: <https://www.postgresql.org/docs/> (дата обращения: 11.05.2025). – Текст: электронный.

51. Обзор баз данных PostgreSQL : сайт / Cnews, 2022 – . – URL: https://market.cnews.ru/news/top/2022-04-25_obzor_baz_dannyh_postgresql (дата обращения: 11.05.2025). – Текст: электронный.

52. ИИ в сельском хозяйстве : сайт / Sber Developer. – Москва : Sber Developer, 2024 – . – URL: <https://developers.sber.ru/help/gigachat-api/ai-agricultural> (дата обращения: 14.05.2025). – Текст: электронный.

53. Цифровые технологии и нейросети в сельском хозяйстве : сайт / It фабрика, 2025 – . – URL: <https://it-fabric.ru/catalog/neyroseti/tsifrovye-tekhnologii-i-neyroseti-v-selskom-khozyaystve/> (дата обращения: 12.05.2025). – Текст: электронный.

54. Использование машинного обучения и ИИ в сельском хозяйстве : сайт / БиоТех2030. – Москва : БиоТех2030, 2025 – . – URL: <http://biotech2030.ru/kak-ii-i-kompyuternoe-zrenie-mogut-sledit-za-rasteniyami/> (дата обращения: 12.05.2025). – Текст: электронный.

ПРИЛОЖЕНИЕ А

Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.13.

Сведения о ВКРБ

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА ПО ПРОГРАММЕ БАКАЛАВРИАТА

«Интеллектуальная система для выявления заболеваний
сельскохозяйственных растений»

Руководитель ВКРБ
д.т.н, доцент
Томакова Римма Александровна

Автор ВКРБ
студент группы ПО-116
Крюков Никита Михайлович

				ВКРБ 21060071.09.03.04.25.015			
				Сведения о ВКРБ			
Авторы работы	Фамилия И.О.	Инициал	Дата	Дет.	Мес.	Минут	
Руководитель	Томакова Р.А.						
Корректор	Чистова А.А.						
				Выпускная квалификационная работа бакалавра			
				ЮЗГУ ПО-116			

Рисунок А.1 – Сведения о ВКРБ

Цель и задачи разработки

Цель настоящей работы – разработка интеллектуальной системы для выявления заболеваний сельскохозяйственных растений, в частности томатов, на основе технологий компьютерного зрения и веб-технологий.

В соответствии с целью работы были сформулированы следующие задачи:

- провести анализ предметной области;
- разработать концептуальную модель интеллектуальной системы для выявления заболеваний томатов;
- спроектировать модель нейронной сети и веб-приложения;
- реализовать и протестировать программную систему.

				ВКРБ 21060071.09.03.04.25.015		
				Цель и задачи разработки		
				Выпускная квалификационная работа бакалавра		
				код УПВ-116		

Рисунок А.2 – Цель и задачи разработки

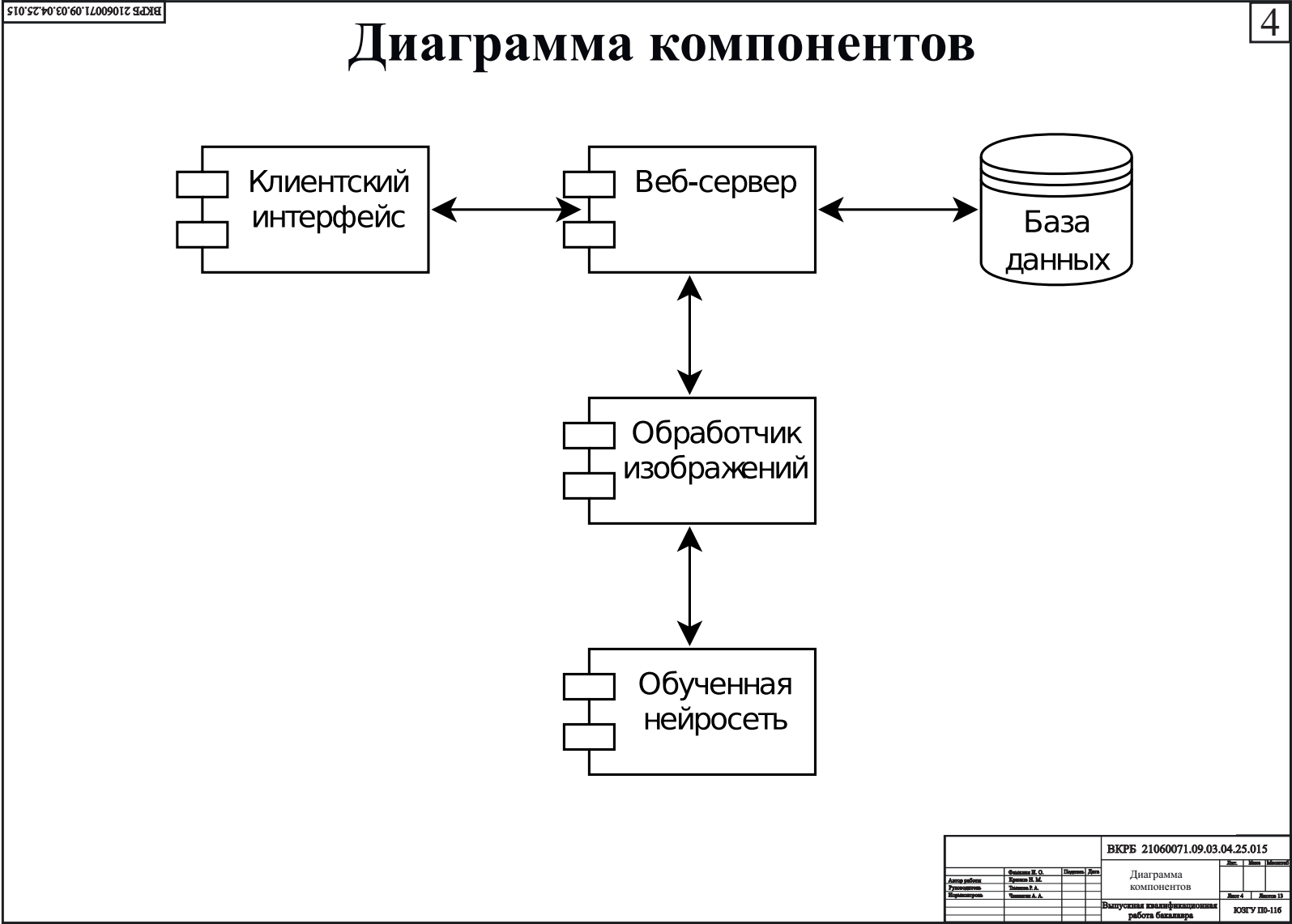


Рисунок А.4 – Диаграмма компонентов

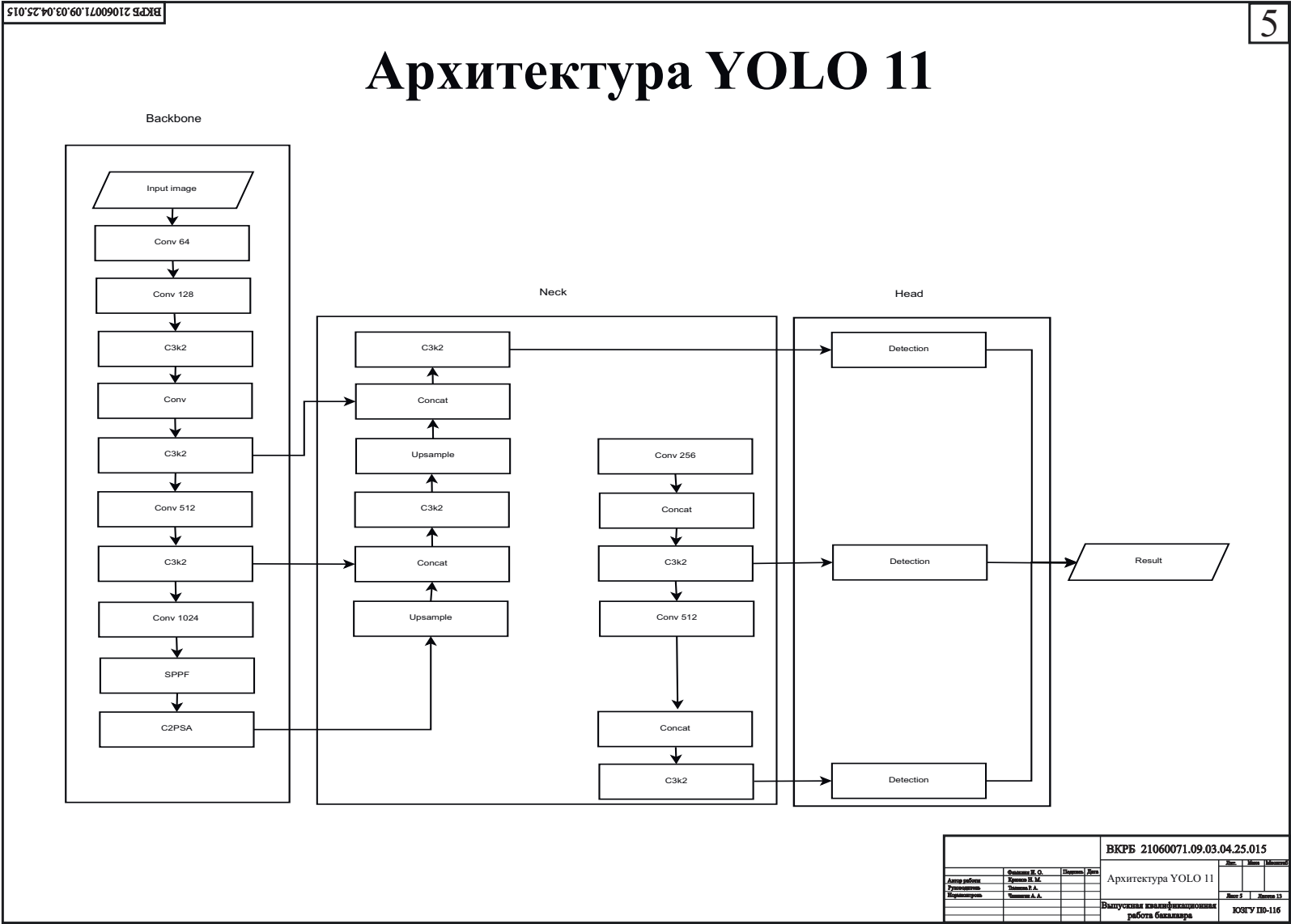
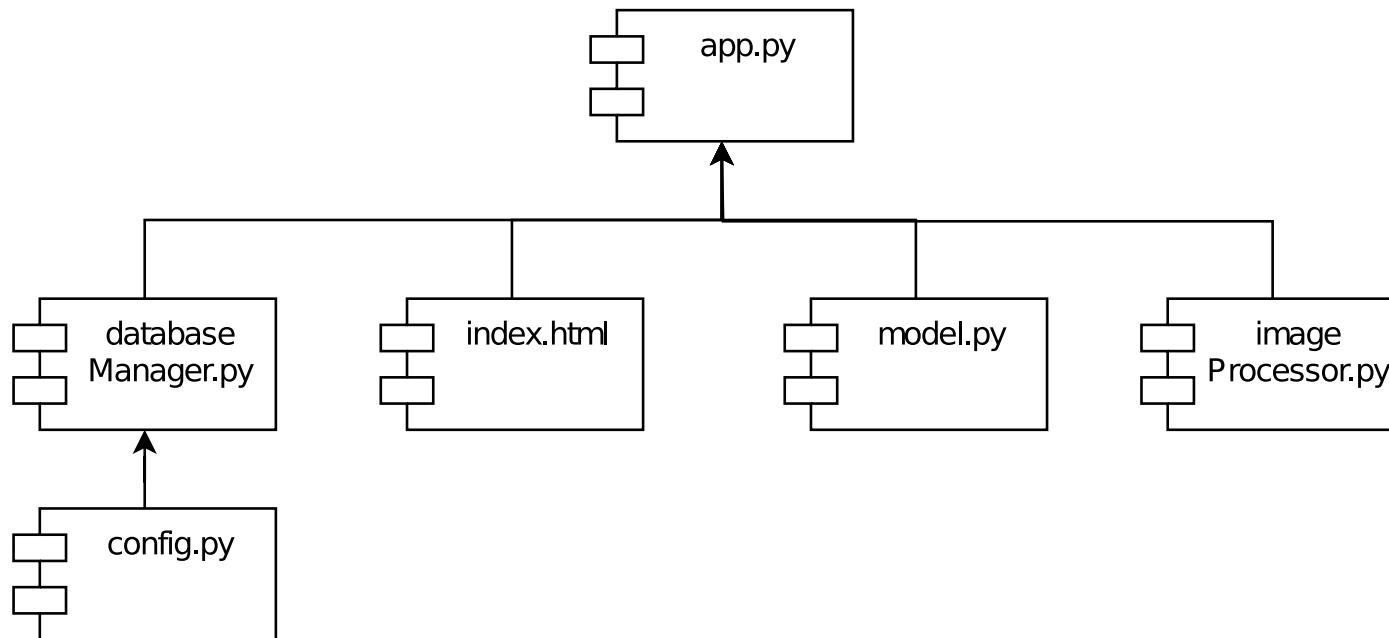


Рисунок А.5 – Архитектура YOLO 11

Структура модулей



				ВКРБ 21060071.09.03.04.25.015			
				Структура модулей	Лист 6	Лист 13	
Автор работы	Филиппов В.О.	Проверено	Дата				
Руководитель	Павлов В.И.						
Корректор	Тимофеев А.А.						
				Выпускная квалификационная работа бакалавра		ЮЗГУ ИО-116	

Рисунок А.6 – Структура модулей

87

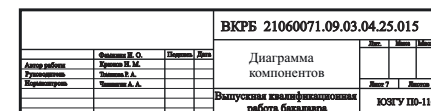
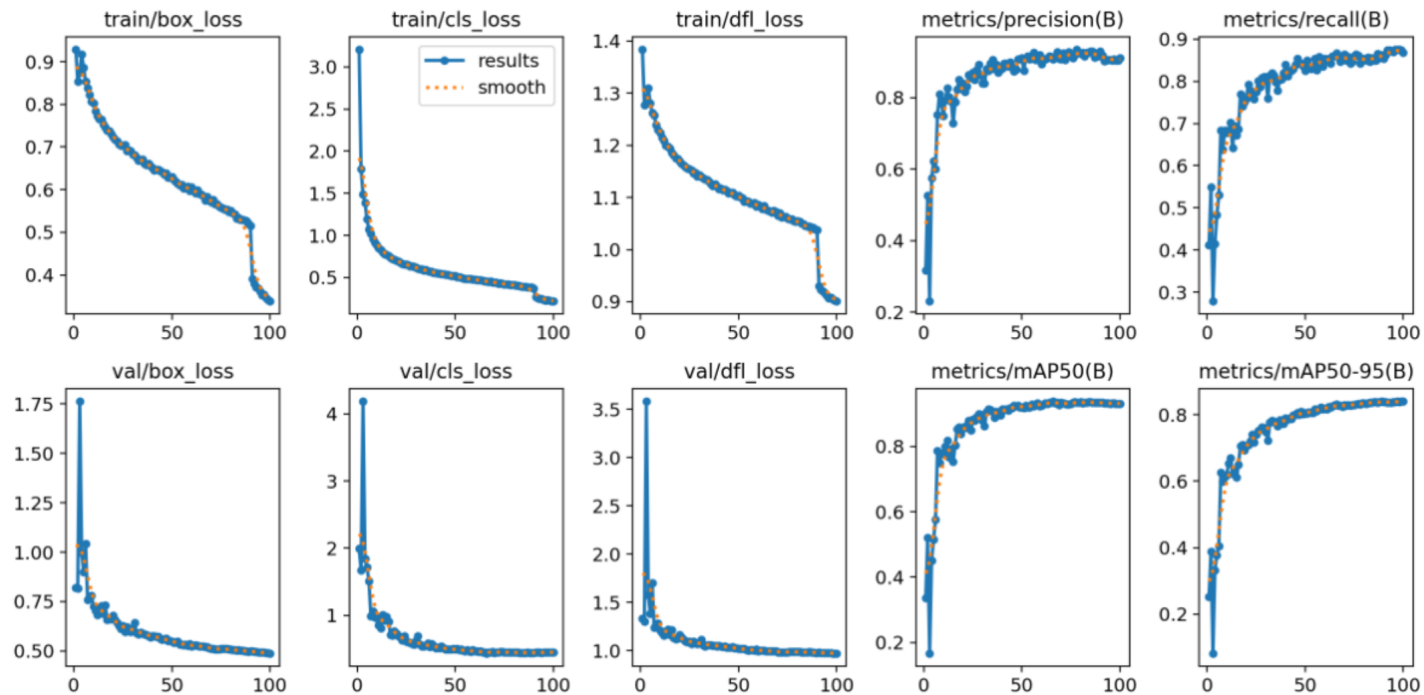


Рисунок А.7 – ER-диаграмма

Динамика метрик обучения



ВКРБ 21060071.09.03.04.25.015				ВКРБ 21060071.09.03.04.25.015		
Автор работы:	Филиппов В.О.	Получено:	Дата:	Динамика метрик обучения		
Руководитель:	Кузнецов В.М.					
Перепроверено:	Чистов А.А.					
	Чистов А.А.			Выпускная квалификационная работа бакалавра		
				ЮЗГУ Пб-116		

Рисунок А.8 – Динамика метрик обучения

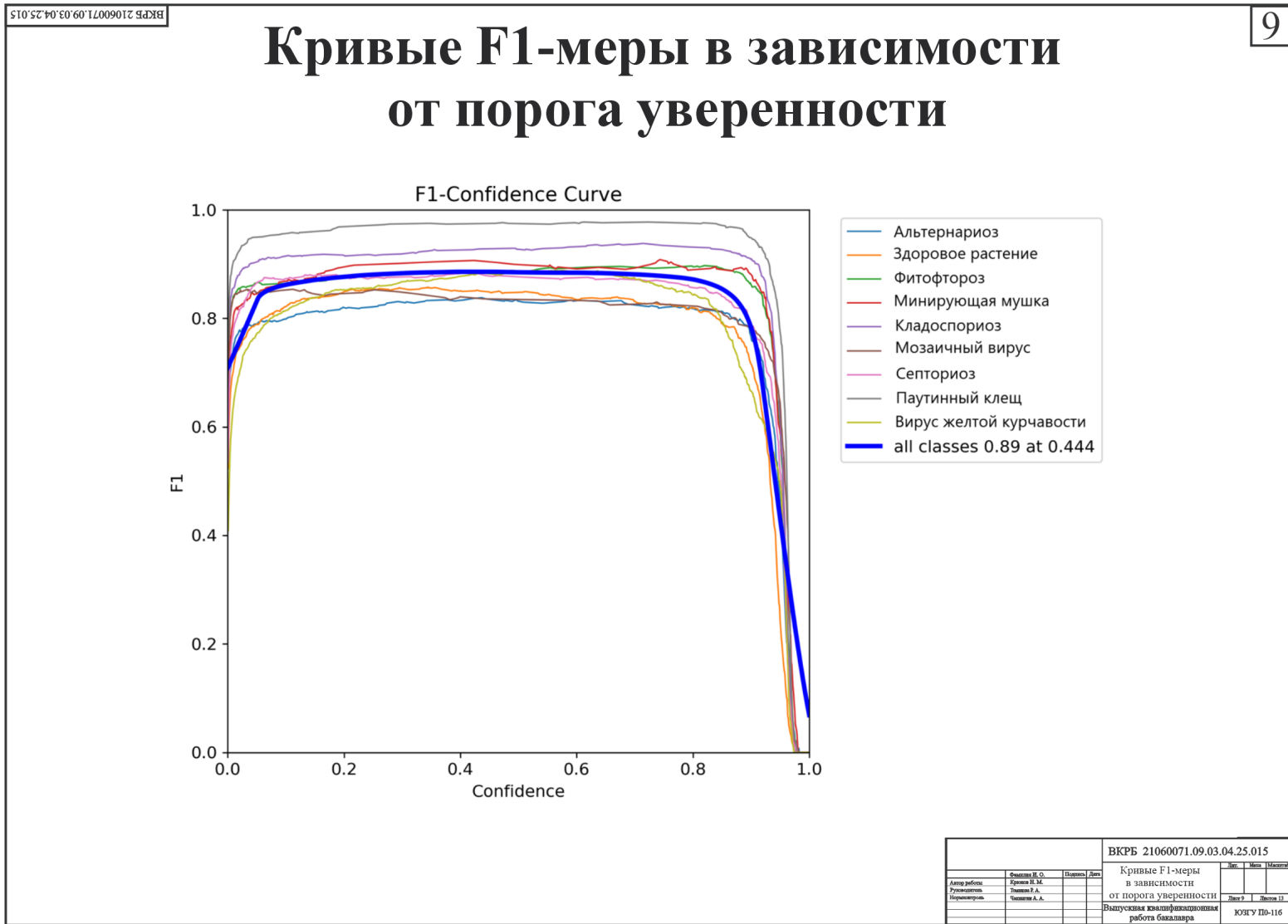


Рисунок А.9 – Кривые F1-меры в зависимости от порога уверенности

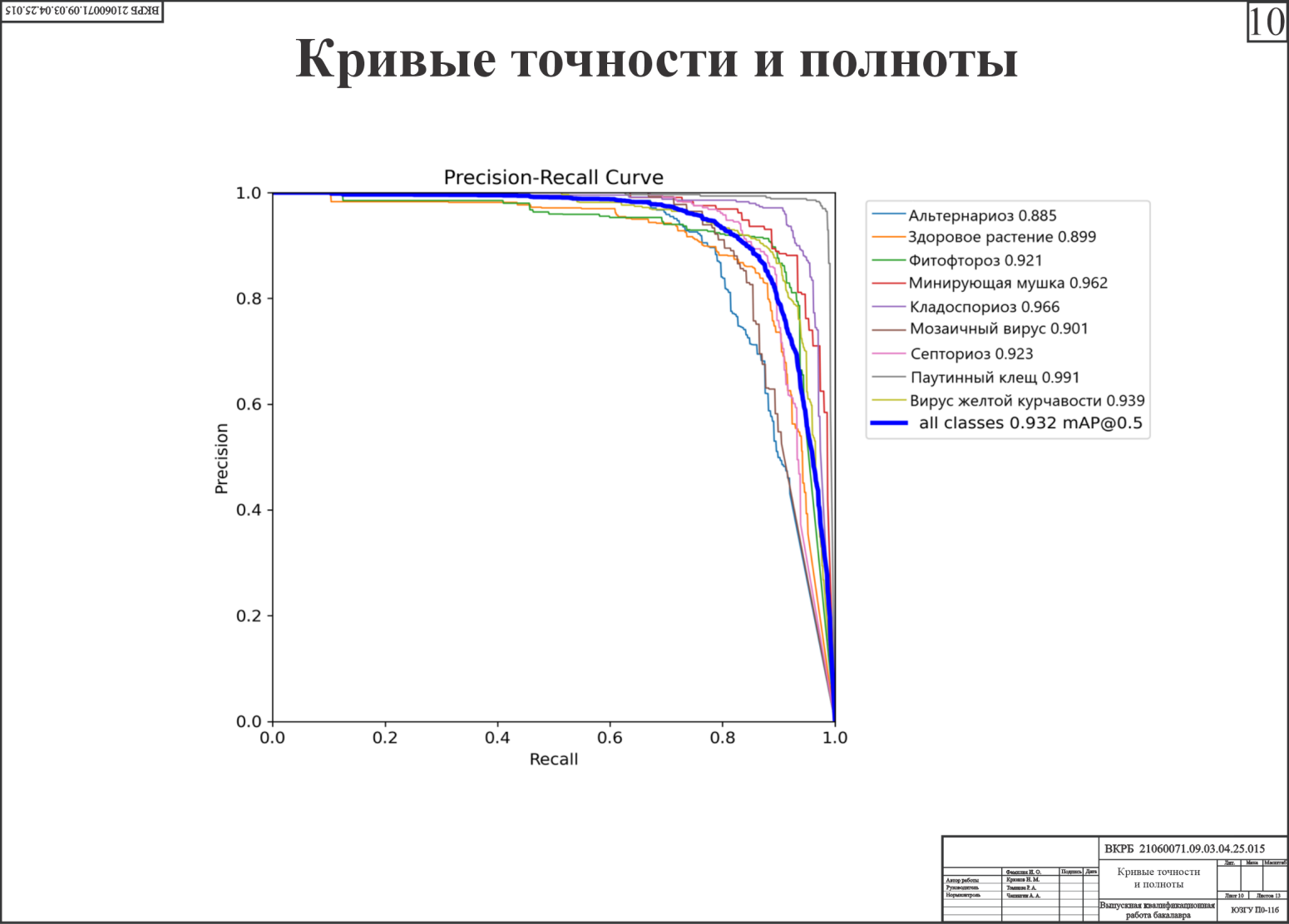


Рисунок А.10 – Кривые точности и полноты



Рисунок А.11 – Нормализованная матрица ошибок

ВКРБ 21060071.09.03.04.25.015

12

Результат анализа фотографии

Обнаруженная болезнь: Альтернариоз (92.98% уверенности)



ВКРБ 21060071.09.03.04.25.015			
Результат анализа фотографии			
Дата работы	Фамилия И. О.	Подпись	Дата
Эксперт	Кузнецов И. М.		
Модератор	Чемпион А. А.		
Выпускная квалификационная работа бакалавра		ЮНУ-ПБ-116	

Рисунок А.12 – Результат анализа фотографии

Заключение

В ходе выполнения данной работы была разработана интеллектуальная система для выявления заболеваний сельскохозяйственных растений, в частности томатов, с использованием технологий машинного обучения и веб-разработки.

Основные результаты работы:

1. Проведен анализ предметной области. Рассмотрены основные заболевания томатов, современные технологий, которые могут использоваться для их диагностики. Проанализированы ограничения традиционных методов обнаружения заболеваний и преимущества применения ИИ.

2. Спроектирована структура интеллектуальной системы. Определены функциональные и нефункциональные требования.

3. Реализована и обучена свёрточная нейронная сеть на базе модели Yolo11. CNN демонстрирует хороший результат, достигая точности mAP50-95 более 0.8.

4. Разработано веб-приложение. Оно позволяет загружать изображения, получать результаты анализа и рекомендации по лечению.

5. Проведено тестирование системы. Оно подтвердило высокую точность модели с F1-мерой 0.89 и средней точностью 0.932 на тестовом наборе данных. Проверена функциональность веб-приложения.

Все требования, объявленные в техническом задании, были полностью реализованы, все задачи, поставленные в начале разработки проекта, были также решены.

Разработанная система представляет собой эффективный инструмент, который может помочь оптимизировать процесс мониторинга здоровья растений и минимизировать потери урожая.

				ВКРБ 21060071.09.03.04.25.015			
Дата работы	Фамилия И.О.	Инициал	Дата	Заключение			
	Применяет	Инициал	Дата				
	Проверяет	Инициал	Дата	Выпускная квалификационная работа бакалавра			
				Итого 10 - 100%			

Рисунок А.13 – Заключение

ПРИЛОЖЕНИЕ Б

Фрагменты исходного кода программы

Модуль trainModel.py

```
1 class YOLOTrainer:
2     def __init__(self, model_path,
3                   data_path,
4                   epochs=100, imgsz=640, batch=16):
5         """
6         Инициализация YOLOTrainer с параметрами обучения.
7
8         Аргументы:
9             model_path (str): Путь к весам модели YOLO
10            data_path (str): Путь к файлу конфигурации данных YAML
11            epochs (int): Количество эпох обучения
12            imgsz (int): Размер изображения для обучения
13            batch (int): Размер пакета (16 для GPU, например, T4)
14        """
15        self.model_path = model_path
16        self.data_path = data_path
17        self.epochs = epochs
18        self.imgsz = imgsz
19        self.batch = batch
20        self.model = None
21        self.train_results = None
22
23    def load_model(self):
24        """Загрузка модели YOLO."""
25        self.model = YOLO(self.model_path)
26
27    def train(self):
28        """Обучение модели YOLOv11 с явными гиперпараметрами."""
29        if self.model is None:
30            self.load_model()
31        device = 'cuda' if torch.cuda.is_available() else 'cpu'
32        results = self.model.train(
33            data=self.data_path,
34            epochs=self.epochs,
35            imgsz=self.imgsz,
36            batch=self.batch,
37            device=device,
38            lr0=0.001,
39            optimizer="AdamW",
40            weight_decay=0.0005,
41        )
42        return results
43
44    def test(self, data_path=None, split="test", batch=16, conf=0.5):
45        """Оценка модели на тестовом наборе данных."""
46        if self.model is None:
47            self.load_model()
48
49        if data_path is None:
```

```

50         data_path = self.data_path
51
52         device = 'cuda' if torch.cuda.is_available() else 'cpu'
53         results = self.model.val(
54             data=data_path,
55             imgsz=self.imgsz,
56             batch=batch,
57             split=split,
58             conf=conf,
59             device=device
60         )
61
62         return results
63
64     def predict(self, source, conf=0.5, save=False):
65         """Выполнение предсказания с использованием модели."""
66         if self.model is None:
67             self.load_model()
68
69         device = 'cuda' if torch.cuda.is_available() else 'cpu'
70
71         results = self.model.predict(
72             source=source,
73             imgsz=self.imgsz,
74             conf=conf, save=save,
75             device=device
76         )
77
78         return results
79
80     def save_model(self, save_path="/content/drive/MyDrive/tomato_leaf_11.pt"):
81         """Сохранение обученной модели на Google Drive."""
82         if self.model is None:
83             return
84
85         self.model.save(save_path)
86
87
88     import matplotlib.pyplot as plt
89     import numpy as np
90     import cv2
91     from ultralytics import YOLO
92     import os
93     import random
94     from pathlib import Path
95     from PIL import Image, ImageDraw, ImageFont
96
97     class YOLOv11Inference:
98         def __init__(self, model_path,
99                     device, class_labels, conf):
100             """
101             Инициализация модели YOLOv11 для выполнения предсказаний.
102

```



```

103     Аргументы:
104         model_path (str): Путь к обученной модели YOLOv11.
105         device (str): Устройство для предсказаний ('cuda:0' или 'cpu').
106         class_labels (list): Список меток классов для заболеваний.
107         conf (float): Порог уверенности для предсказаний -(0.01.0).
108     """
109     if not os.path.exists(model_path):
110         raise FileNotFoundError(f"Модель не найдена: {model_path}")
111     if not class_labels:
112         raise ValueError("Список меток классов не может быть пустым")
113
114     self.device = device
115     self.class_labels = class_labels
116     self.conf = conf
117     self.model = YOLO(model_path).to(self.device)
118
119 def run(self, image_dir, n_ims=15, rows=3):
120     """
121     Выполняет предсказание на случайных изображениях из директории и
122     отображает результаты в Colab.
123
124     Аргументы:
125         image_dir (str): Путь к директории с изображениями растений.
126         n_ims (int): Количество изображений для визуализации.
127         rows (int): Количество строк в сетке визуализации.
128
129     Возвращает:
130         list: Список словарей с результатами предсказаний (путь, рамки,
131             метки, уверенности).
132     """
133     if not os.path.isdir(image_dir):
134         raise FileNotFoundError(f"Директория не найдена: {image_dir}")
135
136     image_extensions = (".jpg", ".jpeg", ".png")
137     image_files = [f for f in Path(image_dir).glob("*") if f.suffix.lower
138                     () in image_extensions]
139
140     if not image_files:
141         raise ValueError(f"В директории {image_dir} нет изображений")
142
143     n_ims = min(n_ims, len(image_files))
144     selected_files = random.sample(image_files, n_ims)
145
146     inference_results = self.model(selected_files, device=self.device,
147                                   verbose=False, conf=self.conf)
148     results_data = self.process_results(inference_results)
149     self.visualize_results(inference_results, n_ims, rows)
150
151     return results_data
152
153 def predict_single(self, image_path):
154     """
155     Выполняет предсказание на одном изображении и возвращает результаты.

```

```

153     Аргументы:
154         image_path (str): Путь к изображению.
155
156     Возвращает:
157         dict: Словарь с результатами предсказания.
158     """
159     if not os.path.exists(image_path):
160         raise FileNotFoundError(f"Изображение не найдено: {image_path}")
161
162     results = self.model(image_path, device=self.device, verbose=False,
163                          conf=self.conf)
164
165     return self.process_results(results)[0] if results else {}
166
167 def process_results(self, results):
168     """
169     Обработывает результаты предсказания, извлекая рамки, метки и
170     уверенности.
171
172     Аргументы:
173         results (list): Результаты предсказания от модели YOLO.
174
175     Возвращает:
176         list: Список словарей с данными о предсказаниях.
177     """
178     results_data = []
179     for r in results:
180         boxes = []
181         for bbox in r.bboxes:
182             box = bbox.xyxy[0].cpu().numpy()
183             class_id = int(bbox.cls[0])
184             confidence = float(bbox.conf[0])
185             boxes.append({
186                 "bbox": [int(box[0]), int(box[1]), int(box[2]), int(box
187                     [3])],
188                 "label": self.class_labels[class_id],
189                 "confidence": confidence
190             })
191         results_data.append({"path": str(r.path), "boxes": boxes})
192     return results_data
193
194 def visualize_results(self, res, n_ims, rows):
195     cols = (n_ims + rows - 1) // rows
196     fig, axes = plt.subplots(rows, cols, figsize=(cols * 5, rows * 5))
197
198     axes = axes.flatten() if isinstance(axes, np.ndarray) else [axes]
199
200     for idx, r in enumerate(res[:n_ims]):
201         img = np.array(Image.open(r.path).convert("RGB"))
202         axes[idx].imshow(img)
203
204         for bbox in r.bboxes:
205             x1, y1, x2, y2 = map(int, bbox.xyxy[0])

```

```

204         rect = plt.Rectangle(
205             (x1, y1), x2 - x1, y2 - y1,
206             fill=False, color='yellow', linewidth=2
207         )
208         axes[idx].add_patch(rect)
209
210         label = f"{self.class_labels[int(bbox.cls[0])]} {float(bbox.
211             conf[0]):.2f}"
212         axes[idx].text(
213             x1, y1 - 10, label,
214             color='yellow', fontsize=12,
215             bbox=dict(facecolor='black', alpha=0.5, edgecolor='none')
216         )
217
218         axes[idx].axis('off')
219
220     for idx in range(len(res), rows * cols):
221         axes[idx].axis('off')
222
223     plt.tight_layout()
224     plt.show()

```

Модуль imageProcessor.py

```

1 from PIL import Image, ImageEnhance
2 import numpy as np
3 import cv2
4 import base64
5
6
7 class ImageProcessor:
8     @staticmethod
9     def _ensure_rgb_image(image):
10         """
11         Проверяет изображение и конвертирует в RGB, если необходимо.
12
13         Args:
14             image: Объект PIL.Image
15
16         Returns:
17             Изображение в формате RGB
18         """
19         if not isinstance(image, Image.Image):
20             raise ValueError("Входной параметр должен быть объектом PIL.Image")
21         if image.mode != 'RGB':
22             image = image.convert('RGB')
23
24         return image
25
26     @staticmethod
27     def _resize_and_pad_image(image, target_size):
28         """
29         Изменяет размер изображения с сохранением пропорций и добавляет
30         отступы.

```

```

30
31     Args:
32         image: Объект PIL.Image
33         target_size: Целевой размер (ширина и высота)
34
35     Returns:
36         Изображение с измененным размером и отступами
37         """
38     width, height = image.size
39     ratio = min(target_size / width, target_size / height)
40     new_width = int(width * ratio)
41     new_height = int(height * ratio)
42     resized_image = image.resize((new_width, new_height), Image.
43         Resampling.LANCZOS)
44
45     padded_image = Image.new('RGB', (target_size, target_size), (0, 0, 0)
46         )
47     padded_image.paste(resized_image, ((target_size - new_width) // 2, (
48         target_size - new_height) // 2))
49
50     return padded_image
51
52 @staticmethod
53 def _enhance_image(image):
54     """
55     Применяет улучшения контрастности, резкости и цвета.
56
57     Args:
58         image: Объект PIL.Image
59
60     Returns:
61         Улучшенное изображение
62         """
63     enhanced_image = ImageEnhance.Contrast(image).enhance(1.0)
64     enhanced_image = ImageEnhance.Sharpness(enhanced_image).enhance(0.5)
65     enhanced_image = ImageEnhance.Color(enhanced_image).enhance(2.0)
66
67     return enhanced_image
68
69 @staticmethod
70 def preprocess_image(image):
71     """
72     Обработывает изображение для предсказания: изменяет размер, добавляет
73     отступы и улучшает качество.
74
75     Args:
76         image: Объект PIL.Image
77
78     Returns:
79         Обработанное изображение PIL.Image
80         """
81     image = ImageProcessor._ensure_rgb_image(image)
82     resized_padded_image = ImageProcessor._resize_and_pad_image(image,
83         target_size=640)

```

```

79         enhanced_image = ImageProcessor._enhance_image(resized_padded_image)
80
81     return enhanced_image
82
83     @staticmethod
84     def resize_for_display(image):
85         """
86         Изменяет размер изображения для отображения в веб-приложении.
87
88         Args:
89             image: Объект PIL.Image
90
91         Returns:
92             Измененное изображение PIL.Image
93         """
94         image = ImageProcessor._ensure_rgb_image(image)
95         target_size = 500
96         resized_image = image.resize((target_size, target_size), Image.
            Resampling.LANCZOS)
97
98         return resized_image
99
100    @staticmethod
101    def _convert_to_bgr_array(image):
102        """
103        Конвертирует изображение в numpy массив и преобразует в BGR.
104
105        Args:
106            image: Объект PIL.Image
107
108        Returns:
109            Массив BGR
110        """
111        image_array = np.array(image)
112        bgr_array = cv2.cvtColor(image_array, cv2.COLOR_RGB2BGR)
113
114        return bgr_array
115
116    @staticmethod
117    def _encode_to_base64(bgr_array):
118        """
119        Кодировать массив BGR в строку base64.
120
121        Args:
122            bgr_array: Numpy массив в формате BGR
123
124        Returns:
125            Строка base64
126        """
127        _, buffer = cv2.imencode('.png', bgr_array)
128        base64_string = base64.b64encode(buffer).decode('utf-8')
129
130        return base64_string
131

```

```

132 @staticmethod
133 def convert_to_base64(image):
134     """
135     Конвертирует изображение в строку base64 для отображения в веб-
        приложении.
136
137     Args:
138         image: Объект PIL.Image
139
140     Returns:
141         Строка base64
142     """
143     bgr_array = ImageProcessor._convert_to_bgr_array(image)
144     base64_string = ImageProcessor._encode_to_base64(bgr_array)
145
146     return base64_string

```

Автор ВКР

(подпись, дата)

Н. М. Крюков

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

Место для диска